

Duty-Free Bits: Projectivizing Garbling Schemes

Nakul Khambhati
nakul@cs.ucla.edu
UCLA, Alpen Labs

Anwesh Bhattacharya
anwesh2@illinois.edu
UIUC

David Heath
daheath@illinois.edu
UIUC

Abstract

Garbling schemes are powerful primitives that enable secure computation between a mutually untrusting *garbler* and *evaluator*. A *projective* garbling scheme is one that encodes the evaluator’s input in a simple bit-by-bit manner. Projective schemes, such as the seminal scheme of Yao, are versatile, as they are naturally compatible with other simple tools, such as 1-out-of-2 oblivious transfer (OT).

There exist garbling schemes that naturally operate over large finite fields, some of which require only efficient information-theoretic (IT) techniques. However, here the evaluator’s input is encoded via an affine function over a large field, so these schemes are not naturally projective, reducing their versatility.

We provide a transformation that efficiently *projectivizes* such schemes. Consider an arithmetic garbling scheme where the evaluator’s input consists of elements from a large prime field. Our symmetric-key-based garbling techniques give a mechanism to translate from Yao-style garbled labels to IT-style garbled labels at cost *proportional to the input and output labels*: crossing the border is duty-free!

We apply our technique to two problems:

- Recent works show that projective garbling schemes solve a problem central to *trust-minimized bridges* for the Bitcoin blockchain. BABE (Garg et al., 2026) and Argo MAC (Eagen and Lai, 2026) give two different approaches. Both works implicitly construct an efficient IT garbling scheme, then use naive bit-decomposition to achieve projectivity. We construct drop-in replacements for both; we improve BABE’s encoding size by 45×, and Argo MAC’s by 20×.
- Our technique implies a non-interactive reduction from vector oblivious linear evaluations (VOLEs) over $\mathbb{F}_p$ to 1-out-of-2 OTs. To our knowledge, ours is the state-of-the-art Minicrypt (plus base OTs) protocol for large field VOLE secure against a malicious receiver. It costs only $O((\lambda + n) \lg p)$ bits.

Keywords: Garbled Circuits, Information-Theoretic Garbling, Projectivity, Bitcoin Bridges, Vector OLE

Contents

1	Introduction	3
1.1	Our Contribution	4
1.2	Applications	5
2	Notation	6
3	Preliminaries	7
3.1	Garbling Schemes	7
3.2	Cryptographic Assumption	8
3.3	Switch Systems	9
3.4	Statistical Smudging	10
4	Overview	10
4.1	Projectivizing Affine Garbling Schemes	10
4.2	Projectivizing Affine Maps	11
4.3	Affine Evaluation via CRT Residues	11
4.4	Application to VOLE	12
4.5	Applications to Bitcoin Bridges	13
5	Projectivizing Garbling Schemes	13
5.1	Projective Partial Garbling Scheme for Affine Maps over $\mathbb{F}_p$	13
5.2	From Affine Encodings to Projective Schemes	14
6	Switch System for Affine Maps	15
6.1	Switch Systems with IT Labels	15
6.2	Helper Switch Systems	20
6.3	Construction of $S_{\text{aff-}\mathbb{Z}_M}$	22
6.3.1	Join-width of $S_{\text{aff-}\mathbb{Z}_M}$	22
7	Applications	23
7.1	Argo MAC Encoding	23
7.2	Embryo: Garbled Scalar Multiplication	24
7.3	$\mathbb{F}_p$ -VOLE Secure Against a Malicious Receiver	27
A	Hash-Optimized $S_{\text{aff-}\mathbb{Z}_M}$	33
B	Elliptic Curve Preliminaries	34
B.1	Notation	34
B.2	Elliptic Curve Addition as a Low-Degree Map	35
C	Concrete Cost Analysis	36

1 Introduction

Garbling schemes are powerful cryptographic primitives that enable a form of privacy-preserving outsourced computation. Consider a program P that takes inputs x and y . Garbling schemes allow two parties—an *evaluator* E with input x and a *garbler* G with input y —to factor computation of P into several steps. In this work, we consider the following flow:

- G uses his input y to sample a *garbled program* $\tilde{P}$, as well as some encoding data e .
- The parties use e to *encode* E ’s input x , yielding some encoding $\tilde{x}$.
- G sends $\tilde{P}$ to E , and E uses x , $\tilde{P}$, and $\tilde{x}$ to locally *evaluate*. This evaluation yields $P(x, y)$.

This flow—where y is known at garbling-time and E holds x in cleartext—is a *partial* garbling scheme; see Section 3.1. The main security property is *privacy*: the pair $(\tilde{P}, \tilde{x})$ should hide G ’s input y from E .

Garbling schemes have numerous cryptographic applications, including constant-round secure two-party and multiparty computation (2PC and MPC) protocols. More generally, these schemes are powerful primitives for building higher-level cryptographic objects. Because of their versatility, garbling schemes enjoy intense research focus, and the literature is rich with techniques for building them. Such techniques trade off with one another in terms of simplicity, efficiency, underlying cryptographic assumptions, and expressivity.

The most well-known garbling technique is that of the seminal work of Yao, which was improved by a long line of work [Yao86, BMR90, NPS99, KS08, PSSW09, KMR14, ZRE15, GLNP18, RR21]. This line of work uses symmetric-key-style assumptions to garble any program expressed as a Boolean circuit C .

One crucial advantage of Yao’s scheme is the *simplicity* of how E ’s input x is encoded. For each bit x_i , G chooses two *labels*, one corresponding to the case where $x_i = 0$, the other for case $x_i = 1$. Encoding E ’s input simply involves choosing the label corresponding to x_i . This property—which the literature sometimes refers to as **projectivity** [BHR12b]—is invaluable when using a garbling scheme to build higher-level objects. For instance, a projective scheme can be straightforwardly transformed into a constant-round 2PC protocol by using 1-out-of-2 oblivious transfer (OT) to allow E to choose her input labels. As a second example, projectivity allows garbling schemes to be used in contexts where the per-wire labels are separately *committed*. This latter compatibility—enabled by the fact that projectivity ensures only a feasible number of commitments are needed—has been recently leveraged to achieve so-called *trust-minimized bridges* for the Bitcoin blockchain; see [LAA⁺25, Che25, Eag25] and Section 7 for more discussion.

The crucial *disadvantage* of Yao’s scheme is the size of its encodings. It is generally agreed that the size of encodings $(\tilde{P}, \tilde{x})$ is the most important performance criterion for garbling schemes, as this size imposes communication complexity for higher-level protocols. To garble a Boolean circuit C , the line of work on Yao’s scheme produces encodings of size $O(|C|\lambda)$ bits, where λ is a computational security parameter, namely the bit-length of encryption keys. This cost is problematic in two ways:

1. The factor- λ blow-up is undesirable. In practice, each AND gate in C imposes an encoding of size ≈ 24 bytes or more [RR21]. When C is large, this blow-up can render the technique impractical.
2. Many natural programs P compile to large Boolean circuits C . For instance, arithmetic operations incur asymptotic blow-up in cost once compiled to low-level Boolean gates, even when using the asymptotically-best-known algorithms [HvdH21].

It is desirable to construct garbling schemes that avoid these efficiency problems.

Amazingly, there exist simple, information-theoretically-secure garbling schemes (IT-GSs) that can avoid both problems above, for certain programs P [IK00, IK02, CFIK03, IW14, App17]. For example, consider the case where the program P takes as input elements from some prime field $\mathbb{F}_p$ and computes a constant-degree *arithmetic formula* (i.e., a circuit with no fan-out) f over the input. In this case, there exists a simple IT-GS whose total encoding size is only $O(|f|)$ $\mathbb{F}_p$ elements, more than a factor λ improvement as compared to Yao’s scheme! While IT-GSs are far more restricted in their expressivity as compared to Yao’s scheme, when they can be applied, they are efficient.

Unfortunately, these IT-GSs have a crucial shortcoming, which is our focus in this work: **IT-GSs are not projective**. Indeed, for IT-GS schemes that operate over finite fields $\mathbb{F}_p$, E ’s input x is now itself a vector of $\mathbb{F}_p$ elements, and encoding each element x_i now involves computing $\mathbf{a}x_i + \mathbf{b}$, where $\mathbf{a}, \mathbf{b}$ are vectors

of $\mathbb{F}_p$ elements chosen by G . This greatly limits the applicability of such techniques. For instance, we can no longer use a single 1-out-of-2 OT to encode each element x_i . One could consider encoding E 's input by replacing OT by *vector oblivious linear evaluation* (vector OLE, VOLE), but VOLE protocols are less primitive and costlier. In addition, one might hope to use a garbling scheme to *build* a VOLE protocol, in which case using VOLE to transmit input labels would be circular. Indeed, we use our techniques to reduce VOLE to OT; see Section 7.

It is thus natural to consider augmenting such IT-GSs with projectivity. Such an upgrade would allow us to enjoy the benefits of IT-GS's smaller encodings together with the flexibility of Yao's scheme. Consider an IT-GS computing some function $f(x, y)$ over $\mathbb{F}_p$. Projectivizing this scheme involves building a new garbling scheme that computes a function f' taking an input x' encoded as bits, rather than x encoded as $\mathbb{F}_p$ elements:

$$f'(x', y) = f(\text{bin-to-}\mathbb{F}_p(x'), y) = f(x, y)$$

Here, x' is now a bitstring, rather than a collection of field elements, and $\text{bin-to-}\mathbb{F}_p$ is a function that reinterprets the bitstring x' as $\mathbb{F}_p$ elements in the natural manner.

Interestingly, the function f' , too, can be securely computed by an IT-GS – the IT-GS takes as input E 's bits x' , each encoded as a field element which happens to be from $\{0, 1\} \subseteq \mathbb{F}_p$. The scheme then computes linear operations over these to recover E 's desired input x , then uses the original IT-GC to compute f .

The above transformation is, to our knowledge and modulo constant factors, the best prior approach for upgrading IT-GSs with projectivity. It is also expensive. It multiplies the size of IT-GS encodings by factor $\lg p$, which is clearly undesirable, especially when working with a large field $\mathbb{F}_p$. Formally, if the original IT-GS has encodings of size S field elements, its projective variant has encodings of size $S \lg p$ field elements.

As one illustrative example of this cost, recent work [GKS⁺26] uses IT-GSs to compute a function $\pi, r \mapsto r \cdot \pi$, where π is E 's element of an elliptic curve group and r is G 's private scalar. That is, the goal is to privately add π with itself r times. Ignoring projectivity, for the specific elliptic curve BN254, this complex operation surprisingly can be solved by an IT-GS encoding of size less than 95KiB. However, [GKS⁺26]'s carefully-tuned garbling scheme produces encodings of size more than 22MiB, a $\approx 240\times$ increase in size. This huge blow-up in size is due primarily to their application requiring *projectivity*.

1.1 Our Contribution

We provide an asymptotically- and concretely-improved technique for upgrading garbling schemes with projectivity. Consider a garbling scheme that computes a function $f(x, y)$, where E 's input x is encoded as elements over a large prime field $\mathbb{F}_p$, and suppose the scheme's encodings $(\tilde{f}, \tilde{x})$ are such that $\tilde{x}$ consists of S field elements, and can be computed from x via component-wise affine functions. Given such a scheme, we construct a new *projective* garbling scheme computing the same function with encodings $\tilde{f}', \tilde{x}'$ such that

1. $|\tilde{f}'|$ is at-most $O((|x|\lambda + S) \lg p)$ bits larger than $|\tilde{f}|$.
2. $|\tilde{x}'|$ consists of $|x|$ Yao-style labels, each of length λ and encoded via a simple selection.

Thus, for example, if $|x|$ is a constant and if $\lambda = O(S)$, our encoding size increases by only $O(S \lg p)$ bits. Our scheme's encodings are also concretely small, see Section 1.2 for a brief discussion.

In fact, our technique can be applied to any garbling scheme that encodes E 's input x in a component-wise affine (over a prime field) manner. That is, E 's input x is treated as a vector over $\mathbb{F}_p$, and encoding E 's input corresponds to computing $\mathbf{a}x_i + \mathbf{b}$ for each x_i . The literature sometimes refers to this property as a *decomposable, affine* garbling scheme (or decomposable affine randomized encoding, DARE). Our scheme is proven secure assuming a circular-correlation robust hash function (a CCR hash function, CCRH). CCR hashes are leveraged extensively in the garbling scheme literature, as they enable security of the celebrated Free XOR optimization [KS08, CKKZ12].

At a technical level, our garbling scheme solves the following problem: Given vectors $\mathbf{a}_i, \mathbf{b}_i \in \mathbb{F}_p^S$ from the garbler and scalar $x_i \in \mathbb{F}_p$ from the evaluator, where x_i is written in binary, compute $\mathbf{a}_i x_i + \mathbf{b}_i$. By computing such an affine map for each element in E 's input, we deliver *all* IT-GC labels to E , which she subsequently uses to evaluate $f(x, y)$. Our scheme computes $\mathbf{a}_i x_i + \mathbf{b}_i$ at communication cost only $O((\lambda + S) \lg p)$ bits. Note that this additive term is *proportional to the size of* the Yao-style input labels and IT-GS output labels! Therefore, we describe this conversion as *duty-free*.

A familiar reader may notice that computing $\mathbf{a}_i x_i + \mathbf{b}_i$ is the ideal functionality of vector OLE (VOLE) over $\mathbb{F}_p$, where G is the VOLE sender and E is the VOLE receiver. Indeed, our scheme immediately implies an $\mathbb{F}_p$ VOLE protocol secure against a malicious receiver.

1.2 Applications

Trust-Minimized Bridges for Bitcoin. Recent works [EL26, GKS⁺26] have demonstrated the relevance of IT-GSs to *trust-minimized bridges* for the Bitcoin blockchain. The central challenge is that Bitcoin’s scripting language is too inexpressive to verify arbitrary computation—such as a Groth16 SNARK proof [Gro16]—on-chain. The BitVM line of work [LAA⁺25, Che25, Eag25] addresses this via an optimistic protocol: G commits to a garbled SNARK verifier off-chain, and if G later submits an invalid proof, E uses the pre-shared garbled circuit to generate a fraud proof. This shifts the verification burden off-chain.

A crucial constraint of this setting is that E’s garbled circuit input must be authenticated on-chain via Lamport signatures. In a Lamport signature on a field element $x \in \mathbb{F}_p$, E reveals, for each bit x_j of x , one of two λ -bit preimages $L_{j,0}$ or $L_{j,1}$. This coincides exactly with a projective input encoding: the revealed preimages serve simultaneously as the Lamport signature on-chain and as the garbled input labels off-chain. Projectivity is therefore not merely convenient, it is mandated by the on-chain authentication mechanism.

Both [EL26] and [GKS⁺26] construct efficient IT-GSs for their respective computations, then projectivize via bit-decomposition, incurring an $\lg p$ -fold blowup in encoding size. BABE [GKS⁺26] garbles a scalar multiplication $(r, \pi) \rightarrow r \cdot \pi$ over BN254, where G holds a private scalar r and E holds a curve point π . By combining with a simple witness encryption scheme for linear pairing relations, [GHK⁺25] show how to verify Groth16 proofs on Bitcoin. Their IT-GS outputs $S = O(\lg p)$ encodings over $\mathbb{F}_p$; after bit-decomposition, the garbled encoding is 22.16 MiB. [EL26] (Argo MAC) garbles the computation of $\kappa = O(\lambda)$ homomorphic MAC components on a curve point, each a low-degree polynomial evaluation on the point’s affine coordinates; their IT-GS has $S = O(\kappa)$ encoding elements, and after bit-decomposition produces an encoding of 6.8 MiB.¹

Our technique serves as a drop-in replacement for the projectivization step in both works. For scalar multiplication, our construction—which we call **Embryo**—reduces the garbled encoding to ≈ 500 KiB, a $45\times$ improvement. For Argo MACs, we reduce the encoding to ≈ 355 KiB, a $20\times$ improvement. We achieve this reduction in size *without increasing computation*. **Embryo** incorporates an input validity check at a cost of only $O(\lg p)$ bits; BABE handles this check via a Yao-style garbled circuit, incurring unnecessary overhead. Full constructions are presented in Sections 7.1 and 7.2, and concrete costs are analyzed in Appendix C.

Vector OLE Over Large Fields. Oblivious linear evaluation (OLE) and vector OLE (VOLE) can be understood as generalizations of OT. In OLE, a sender holds a, b and a receiver holds x ; as output, the receiver obtains $ax + b$. Viewed another way, the parties ultimately hold additive secret shares of ax . VOLE generalizes this to the setting where the sender holds two *vectors* $\mathbf{a}, \mathbf{b}$, and the receiver outputs $\mathbf{a}x + \mathbf{b}$. VOLE and OLE are well-studied primitives, due to their numerous applications. For instance, VOLE is a building-block for achieving arithmetic MPC protocols [ADI⁺17], Zero Knowledge proofs [BDSW23], and oblivious pseudorandom function/private set intersection protocols [RS21].

OLE was classically solved by the protocol of [Gil99], which computes a secret-shared multiplication ax via the schoolbook algorithm, where one OT is used to compute ax_i for each bit x_i . When values are in a ring $\mathbb{Z}_m$, this basic approach incurs cost $O(\lg^2 m)$ bits per OLE. [CDK⁺22] used the Chinese Remainder Theorem (CRT) to improve this for rings $\mathbb{Z}_m$ where m is a primorial, at cost $O(\lg m \lg \lg m)$ bits per OLE.

Over the last decade, VOLE greatly improved, thanks to the advent of exciting work on pseudorandom correlation generators (PCGs). PCGs led to random VOLE protocols whose total cost is *sublinear* in the length of the vectors [BCGI18, BCG⁺19]. While these results are remarkable, they rely on strong cryptographic assumptions, such as variants of Learning Parity with Noise (LPN). This reliance means these techniques are expensive, in the sense that vectors need to be relatively large before asymptotic advantage kicks in, and they also incur high computational cost.

It is thus desirable to also achieve OLE/VOLE from simple and efficient Minicrypt primitives. While such primitives seem insufficient to get strongly sublinear cost, they can be more efficient in some settings. For instance, SoftSpokenOT generalized the classic IKNP OT extension protocol, using Minicrypt primitives

¹The size of this encoding depends on the number κ of MAC components generated and the underlying elliptic curve group. We compare with $\kappa = \lceil \lg p / \lg 7 \rceil$ (see [EL26] for more details) and the BN254 elliptic curve group.

to reduce VOLE over small (i.e., poly-sized) fields to OT, and demonstrated best-in-class performance under many network settings [IKNP03, Roy22].

Minicrypt techniques for OLE/VOLE over large fields $\mathbb{F}_p$ have also begun to emerge. A natural goal here is to achieve cost $O(\lg p)$ bits per OLE. [DHIM25] achieved a maliciously-secure reduction from OLE to OT with cost $\tilde{O}(\lg p)$ bits, where $\tilde{O}$ hides $\text{poly}(\lg \lg p)$ factors. Very recently, [CK26] produced a similar reduction from OLE to OT with cost $\lg p \cdot 2^{O(\log^* \lg p)}$ communication and $O((\log^* \lg p)^2)$ rounds, where $\log^*$ denotes the extremely-slow-growing iterated logarithm function.

Our technique immediately implies a Minicrypt reduction from large field VOLE to OT where each OLE incurs amortized cost $O(\lg p)$ bits. Namely, we *non-interactively* reduce n VOLEs to $\lg p$ OTs in zero rounds, at cost $O((\lambda + n) \lg p)$ bits. Indeed, recall that computing $\mathbf{a}x + \mathbf{b}$ for x expressed in bits is essentially our main technical challenge. Our technique is naturally secure against a malicious receiver, because our reduction is non-interactive. Namely, in the OT-hybrid model, the evaluator (aka receiver) simply sends its input x to the OT functionality, and from there it sends no further messages, so security is trivially implied by the privacy of our garbling scheme.

Our technique is somewhat reminiscent of a VOLE-to-OT reduction recently demonstrated by [Kon25]. Like ours, this recent work also performs a VOLE-to-OT reduction that leverages CRT. In particular, [Kon25] uses SoftSpokenOT to perform small-field VOLE over various CRT residues.

Our VOLE improves over [Kon25]’s in several ways. First, ours is naturally non-interactive, whereas theirs needs to apply an extra range check proof to guard against a malicious receiver. So, for instance, the [Kon25] approach cannot be used to build a garbling scheme for our Bitcoin application. Second, ours has lower cost; theirs incurs cost $O((\lambda \lg p / \lg \lg p + n) \lg p)$, whereas ours costs $O(\lambda + n) \lg p$. This is particularly relevant when the number of required VOLE correlations is relatively low. As a final small advantage, our approach reduces to roughly $2\times$ fewer OTs; this is because in our approach, the receiver inputs their scalar x as a field element, whereas in [Kon25] the receiver inputs x as an element of some CRT-friendly ring, which must be more than twice the bit-length of the desired prime field. We formalize this result in Theorem 7.3.

2 Notation

- We write λ for a computational security parameter (e.g. the length of hash outputs or encryption keys).
- We write ρ for a statistical security parameter.
- $\mathbb{F}_p$ denotes a prime field. We often use $n := \lg p$ to denote the bit-length of field elements. In our Bitcoin application, $n = O(\lambda)$.
- We use $\lg$ to denote the *ceiled* base-two logarithm: $\lg(\cdot) = \lceil \log_2(\cdot) \rceil$.
- G denotes the garbler (referred to as “he”); E denotes the evaluator (referred to as “she”).
- Recall, we work in terms of an underlying garbling scheme that encodes E ’s input as field elements. We use S to denote the total number of field elements in this encoding.

CRT parameters. Let $p_1 < p_2 < \dots$ denote the sequence of primes ($p_1 = 2$). Fix n , and define $M := \prod_{t=1}^T p_t$ to be the first primorial satisfying $\lg M \geq 2n + \rho$, and set $m := \lg M$. By the prime number theorem, $p_T = \Theta(m) = \Theta(n)$ and $T = \Theta(m / \log m) = \Theta(n / \log n)$. We write $\mathbb{Z}_M \cong \mathbb{Z}_{p_1} \times \dots \times \mathbb{Z}_{p_T}$ for the corresponding product ring via the Chinese Remainder Theorem.

Representations. For $x \in \{0, \dots, 2^n - 1\}$, we write $\text{bin}(x) = (x_0, \dots, x_{n-1}) \in \{0, 1\}^n$ for its binary decomposition, so that $x = \sum_{j=0}^{n-1} 2^j x_j$. Conversely, for a bitstring $\mathbf{x} \in \{0, 1\}^n$, we write $\text{bin-to-}\mathbb{F}_p(\mathbf{x}) := \sum_{j=0}^{n-1} 2^j x_j \bmod p \in \mathbb{F}_p$. Also, we denote by $\mathbf{x} = (x_1, \dots, x_{|\mathbf{x}|})$ a vector of ring/field elements with $|\mathbf{x}|$ being the length of the vector.

3 Preliminaries

3.1 Garbling Schemes

We use the following definition of a *partial* garbling scheme, which is derived from definitions in [BHR12b, IW14]. The notion of a *partial* scheme—as opposed to the “full security” notion—corresponds to the fact that the evaluator E knows some of the input to the garbled program in cleartext. This notion maps well to our setting of augmenting IT-GSs with projectivity, as our main goal is to construct a garbling scheme that generates an encoding of E ’s input, which E knows.

Definition 1 (Partial Garbling Scheme). A **partial garbling scheme** for a function $f : \mathcal{X} \times \mathcal{Y} \rightarrow \mathcal{Z}$ is a triple of algorithms $(\text{Garb}, \text{Enc}, \text{Eval})$, where:

- **Garb** takes a security parameter 1^λ , and a garbler input $y \in \mathcal{Y}$. It outputs an encoding $\tilde{f}$, and an input encoding string e .
- **Enc** takes an input encoding string e and an evaluator input $x \in \mathcal{X}$. It outputs an encoding $\tilde{x}$.
- **Eval** takes a program encoding $\tilde{f}$ and an input encoding $\tilde{x}$. It outputs a value $z \in \mathcal{Z}$.

The scheme is **correct** if for all inputs $x \in \mathcal{X}, y \in \mathcal{Y}$ the following holds with overwhelming probability in λ :

$$\Pr \left[f(x, y) = z \mid \begin{array}{l} (\tilde{f}, e) \leftarrow \text{Garb}(1^\lambda, y) \\ \tilde{x} \leftarrow \text{Enc}(e, x) \\ z \leftarrow \text{Eval}(\tilde{f}, \tilde{x}) \end{array} \right] > 1 - \text{negl}(\lambda)$$

The scheme is **private** if there exists a simulator $\mathcal{S}$ such that for all inputs $x \in \mathcal{X}, y \in \mathcal{Y}$, the following are indistinguishable in λ :

$$\left\{ (\tilde{f}, \tilde{x}) \mid \begin{array}{l} (\tilde{f}, e) \leftarrow \text{Garb}(1^\lambda, y) \\ \tilde{x} \leftarrow \text{Enc}(e, x) \end{array} \right\} \approx \left\{ \mathcal{S}(1^\lambda, x, f(x, y)) \right\}$$

We say the scheme is **statistical** if the above indistinguishability holds even against unbounded adversaries (i.e., the ensembles are statistically close).

The scheme is **affine** if $\mathcal{X} \subset R^{|x|}$ for some finite ring R and for fixed input encoding string e , $\text{Enc}(e, x)$ is component-wise affine. The scheme is **projective** if the scheme is affine and $R = \{0, 1\}$.

As compared to the definition in [IW14], the above definition provides separate **Garb** and **Enc** procedures. The former generates a garbled program, the latter encodes the evaluator’s input as labels. By contrast, [IW14] has a single encoding step that combines these two procedures. Ultimately, these two notions are not meaningfully different; we elect to make explicit a separate garbling step because in our main application, the garbled circuit is conveyed before input labels.

Some works distinguish between **affine** (meaning **Enc** is affine over $R' = R^{|x|}$) and **decomposable** (meaning **Enc** acts independently on each input component). Our definition of affine already subsumes decomposability, since we require **Enc** to be component-wise affine over $R^{|x|}$.

Adaptive security. The above notion of privacy is *selective*: The evaluator’s input is chosen *independently* of the garbled program $\tilde{f}$. There is a natural, more challenging setting where we separate execution into an *offline phase*—when the garbled program is sent—and an *online phase*—when the evaluator chooses its input. This setting requires that we change our security notion such that the adversarial evaluator may choose its input x after having seen the garbled program $\tilde{f}$.

This *adaptive* notion of security is crucial to many applications—such as our Bitcoin application—where the garbler should send the garbled program in advance of the evaluator choosing its input. Adaptive security is highly studied, primarily because it seems hard to achieve from plain-model symmetric-key cryptography [BHR12a, HJO⁺16, JW16, JSW17, GS18, KKPW21, AAK25].

On the other hand, adaptive security is quite easy to achieve in an idealized model, such as the random oracle model [BHR12a, GYW⁺23, BHKO23]. One strategy is to simply encrypt a selectively-secure garbled

program with the output of a random oracle applied to some uniform key. Then, in the online phase, the garbler simply transmits the key. This automatically upgrades any selectively-secure garbling scheme to an adaptively-secure one; see [BHR12a] for a definition of security and details on the transformation from selective to adaptive security.

IT-GS Garbling Schemes. As mentioned in Section 1, there exist efficient information-theoretic garbling schemes for certain classes of programs. We here describe one such technique, needed for our Bitcoin application:

Theorem 3.1 (IT-GS for Sums of Monomials [CFIK03, IK02, AIK11, IW14]). *Let R be a finite ring. Consider a function $f: R^{|x|} \times R^n \rightarrow R$ of the form*

$$f(x, y) = \sum_{i=1}^n \left(y_i \cdot \prod_{j \in S_i} x_j \right),$$

where each $S_i \subseteq [|x|]$ is a multiset i.e. with repeated indices allowed. There exists a statistical affine partial garbling scheme PGS_{poly} for f with $\tilde{f} = \emptyset$ and $|\tilde{x}| = \sum_{i=1}^n |S_i|$ ring elements.

Theorem 3.1 can be viewed as a special case of [IW14]’s IT-GS for arithmetic branching programs (ABPs). Each monomial $y_i \cdot \prod_{j \in S_i} x_j$ corresponds to a chain in an ABP: a length- $|S_i|$ path of evaluator-labeled edges terminating in a single garbler-labeled edge into the sink. Because the evaluator knows x in cleartext, she routes herself through these chains without any garbled program, therefore $\tilde{f} = \emptyset$. The encoding size $\sum_i |S_i|$ is precisely the total number of public edges (equivalently, the number of internal vertices, on a path graph) across all chains.

3.2 Cryptographic Assumption

We prove our garbling scheme secure under the assumption of a circular correlation robust hash (CCRH) function. CCRHs are relatively ubiquitous in the garbled circuit literature, due to their importance to the Free XOR optimization [KS08, CKKZ12]. Roughly, a CCRH produces random-looking output, under the assumption that inputs are keyed by some correlation $\Delta \in \{0, 1\}^\lambda$, even when the output of the CCRH is used to encrypt values that involves Δ .

Definition 2 (Circular Correlation Robustness). *Let S denote a set of nonces. Let $H: \{0, 1\}^\lambda \times S \rightarrow \{0, 1\}^\lambda$ be a poly-time computable function. We define the following helper function:*

$$\begin{aligned} \text{circ} : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \times S \times \{0, 1\} &\rightarrow \{0, 1\}^\lambda \\ \text{circ}(\Delta, x, i, b) &= H(x \oplus \Delta, i) \oplus b\Delta \end{aligned}$$

Consider the following two probability ensembles of oracles:

$$\left\{ \text{circ}(\Delta, \cdot, \cdot, \cdot) \mid \Delta \in_{\$} \{0, 1\}^\lambda \right\} \quad \left\{ \text{rand}(\cdot, \cdot, \cdot) \mid \text{rand} : \{0, 1\}^\lambda \times S \times \{0, 1\} \rightarrow \{0, 1\}^\lambda \text{ is uniform} \right\}$$

*We say a sequence of queries to such an oracle is **legal** if no two queries use the same nonce i . We say that H is **circular correlation robust** if no poly-time adversary issuing legal queries can distinguish the above two ensembles.*

[Hea24] showed that the above definition straightforwardly generalizes to a notion of CCR hashes where the encrypted value Δ is arithmetic over $\mathbb{Z}_{2^k}$ for some k :

Lemma 3.1 (CCRH Arithmetic Extension [Hea24]). *Let $H: \{0, 1\}^\lambda \times S \rightarrow \{0, 1\}^\lambda$ be a CCR hash function. Then for any (polynomially-large) k , there exists $H_k: \{0, 1\}^\lambda \times S \rightarrow \mathbb{Z}_{2^k}^\lambda$ such that H_k is CCR over outputs in $\mathbb{Z}_{2^k}$. Namely, the indistinguishability of Definition 2 holds for the following arithmetic extension of circ :*

$$\begin{aligned} \text{circ}_k : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \times S \times \mathbb{Z}_{2^k} &\rightarrow \mathbb{Z}_{2^k}^\lambda \\ \text{circ}_k(\Delta, x, i, b) &= H'(\Delta, i) + b\Delta \end{aligned}$$

In the expression $b\Delta$, each entry of $\Delta \in \{0, 1\}^\lambda$ is treated as an element of $\mathbb{Z}_{2^k}$ in the natural manner. H' issues k black-box calls to H .

In our garbling scheme, we use a CCRH H to encrypt group elements from various groups. These encryptions fall into two types:

- We use H to encrypt Yao-style labels holding values over some ring $\mathbb{Z}_{2^k}$. These labels involve a correlation Δ . When we use H in this way, we mean to apply Lemma 3.1 to construct a CCRH H_k that hides Δ .
- We use H to encrypt IT-GS-style labels holding values over some arbitrary finite group. When the output of H is used in this way, we mean to use the randomness from the output of H to uniformly sample a group element.

3.3 Switch Systems

Switch systems were introduced by [Hea24] as a computational model that captures much of the expressivity of symmetric-key-based garbling. We recall the model and its key capabilities; we extend the framework in Section 6 to suit our constructions. Looking forward, a main contribution of our work is to demonstrate that switch systems are naturally compatible with IT-GS-style garbled labels, and to demonstrate that this compatibility can lead to non-trivial constructions, in particular our projectivization technique.

A switch system is defined over a collection of *wires*, each carrying a value from some ring or group. Rather than specifying a one-directional computation from inputs to outputs—as Boolean circuits do—a switch system specifies local *constraints* on wires. An evaluator satisfies these constraints in whatever order is convenient given the current state of the computation, propagating known wire values to unknown ones. This additional flexibility, relative to Boolean circuits, is precisely what enables efficient garbling of non-Boolean operations. The two principal gate types are as follows.

Switches. A switch takes a data wire x and a control wire $\text{ctrl} \in \{0, 1\}$ and produces an output data wire y . When $\text{ctrl} = 0$, the switch *closes*, enforcing the constraint $x = y$ and propagating whichever of the two is already defined; when $\text{ctrl} = 1$, the switch is open and imposes no constraint. An important requirement of switches is that the evaluator must know ctrl in cleartext—she acts conditionally on its value. In a secure switch system, all switch controls must therefore either be public inputs or be simulatable from them, so they don’t leak information about the garbler’s private input. Looking forward, in our construction, all control values can be inferred from E’s input x . Switch gates contribute *zero communication* when garbled.

Joins. A join $x \bowtie y$ unconditionally enforces the equality $x = y$. Joins are necessary because the switch system model assigns each switch a fresh output wire; expressing that two such outputs are equal requires an explicit join. In the original model [Hea24] each join gate on a wire over a group $\mathbb{G}$ contributes $\lambda \lg |\mathbb{G}|$ bits to the garbled program. The total communication of a garbled switch system is therefore captured by its total *join width*, i.e. the number of bits passing through all join gates.

Affine gates. Switch systems also support affine operations. Following the Free-XOR technique [KS08], these are communication-free.

One-hot scaling. A key example that illustrates the power of switch systems is that of *one-hot scaling* [HK21, Hea24]. A one-hot vector $h \in \{0, 1\}^n$ has exactly one entry equal to 1; scaling h by a scalar s produces a vector whose i -th entry equals s if $h_i = 1$ and 0 otherwise. Naively, a garbled Boolean circuit for this operation would cost $O(n\lambda)$ bits. In a switch system, one attaches to each index i a switch controlled by h_i : when $h_i = 0$, the switch closes and forces the i -th output to 0; the single “active” output is then resolved by one join enforcing that the sum of all outputs equals s . This reduces the cost to a single join—just $O(\lambda)$ bits—regardless of n . One-hot scaling is a central primitive underlying our constructions in Section 6.

Arithmetic garbling. Building on these ideas, [Hea24] used switch systems to garble arithmetic circuits over $\mathbb{Z}_{2^k}$ from circular correlation robust hashes, achieving garbled gate size $O(k\lambda)$ bits per gate, in contrast to the $O(k^2\lambda)$ bits required by schoolbook multiplication. In [Hea24]’s framework, every wire carries a λ -length *control-friendly* label of the form $X + x \cdot \Delta$, where $X \in \mathbb{Z}_{2^k}^\lambda$ is a per-wire one-time-pad mask and

$\Delta \in \{0, 1\}^\lambda$ is a global Free-XOR correlation. This format is required for switches to hash-translate between labels on their two data wires. Several building-block switch systems from [Hea24]—including *bin-to-hot*, *bin-to-word*, and *word-to-ring*—are used directly in our constructions.

IT-friendly switch systems. Our setting introduces a complication: we need wires that carry IT-GS-style labels—single elements of a finite group serving as one-time-pad encodings—rather than length- λ vectors. Passing such labels through [Hea24]’s framework as-is would force a λ -fold blowup in communication, eliminating the efficiency advantage of IT-GSs. We resolve this by introducing a distinction between *control-friendly* and *non-control-friendly* (plain) wires. Control-friendly wires behave exactly as in [Hea24]; plain wires carry short group-element labels and are permitted wherever switch-control security is not required. A join between plain wires over a group $\mathbb{G}$ costs only $\lg |\mathbb{G}|$ bits—no λ factor—allowing IT-GS labels to flow through the switch system without incurring additional overhead. We also generalize [Hea24]’s $\mathbb{Z}_{2^k}$ -specific gate types to *homomorphism gates* and *subgroup gates* that operate over arbitrary finite groups, encompassing the original gates as special cases. We present the full formal treatment of this extended framework and its associated garbling scheme in Section 6.

3.4 Statistical Smudging

Looking forward, our construction achieves arithmetic operations over a field $\mathbb{Z}_p$ by lifting to a CRT-friendly ring $\mathbb{Z}_M$. By setting M large enough, we ensure arithmetic in $\mathbb{Z}_M$ will not wrap around, and output values will be congruent to their value modulo p . However, we will still ultimately need to reduce modulo p . To do this, we simply “outsource” to E , who performs modular reduction locally. This involves revealing values x to E that are written modulo $\mathbb{Z}_M$, whereas it is only secure to reveal $x \bmod p$.

That is, a naive version of our construction would reveal to E $x + rp \bmod M$, where it would not be possible to simulate quantity r . Therefore, we use a standard trick: before we reveal $x + rp \bmod M$, we add some large random multiple μ of p to it, such that E instead learns $x + (r + \mu)p \bmod M$. It is a well-known fact that μ statistically hides r . That is, the quantity $x + (r + \mu)p$ is statistically close to one that does not include the insecure overflow r . We formalize this well-known fact:

Lemma 3.2 (Smudging Lemma). *Let B be a positive integer and let e be an arbitrary random variable supported on $\{0, \dots, B - 1\}$. Let $\mu \in_{\$} \{0, \dots, B \cdot 2^\rho - 1\}$ be chosen uniformly and independently of e . Then the statistical distance between the distributions of μ and $\mu + e$ is at most $2^{-\rho}$.*

4 Overview

In Section 1 we motivated the need to augment information-theoretic garbling schemes (IT-GSs) with *projectivity*—where E ’s input to the garbling scheme is encoded bitwise. For each bit x_i , G chooses two *labels*, one corresponding to the case where $x_i = 0$, the other for case $x_i = 1$. In a projective garbling scheme, encoding E ’s input simply involves choosing the label corresponding to x_i . For functions admitting simple IT-GSs, this yields highly efficient constant-round 2PC protocols: the IT-GS is first projectivized, after which 1-out-of-2 OT enables E to obviously select her input labels.

4.1 Projectivizing Affine Garbling Schemes

Consider a garbling scheme that computes a function $f : \mathcal{X} \times \mathcal{Y} \rightarrow \mathcal{Z}$. Projectivizing f amounts to building a new garbling scheme that instead computes a function $f' : \{0, 1\}^{n'} \times \mathcal{Y} \rightarrow \mathcal{Z}$ which accepts E ’s input as n' bits instead of the native representation of $\mathcal{X}$.

Our projectivization technique can be applied to any garbling scheme that encodes E ’s input in a component-wise affine manner over a ring $\mathbb{Z}_M$ for arbitrary modulus M . Indeed, G ’s input need not even belong to the same domain as that of E . For simplicity, assume $\mathcal{X} = \mathbb{F}_p^{|\mathbf{x}|}$ where $\mathbf{x}$ is the evaluator’s input. The corresponding projectivized f' will then be $f' : \{0, 1\}^{n \cdot |\mathbf{x}|} \times \mathcal{Y} \rightarrow \mathcal{Z}$, where $n = \lg |\mathbb{F}_p|$. Concretely,

writing $x_{i,j}$ for the j -th bit of E 's i -th input element, the scheme first reconstructs each field element

$$x_i = \sum_{j=0}^{n-1} 2^j \cdot x_{i,j} \bmod p,$$

and then evaluates f . Assume that the original garbling scheme produced encodings $(\tilde{f}, \tilde{\mathbf{x}})$ where $\tilde{\mathbf{x}}$ consists of S elements of $\mathbb{F}_p$. Our projectivized garbling scheme produces a garbled program $\tilde{f}'$ of size

$$|\tilde{f}'| = |\tilde{f}| + O((|\mathbf{x}|\lambda + S) \lg p)$$

bits. Compared to the bit decomposition approach described in Section 1, our saving is a factor of $\lg p$ whenever $S = \Omega(|\mathbf{x}|\lambda)$. This garbling scheme can be viewed as a method to convert Yao-style projective encodings to IT encodings that are affine over some field $\mathbb{F}_p$. The additive cost we pay for this conversion is, in fact, *proportional to the size of the input projective encodings and the output affine encodings!* Therefore, we describe this conversion as *duty-free*.

4.2 Projectivizing Affine Maps

Recall that for our transformation, we only require that the base garbling scheme for f encodes inputs component-wise via affine functions:

$$\mathbf{a}_i x_i + \mathbf{b}_i, \quad i \in [|\mathbf{x}|].$$

Each such function takes as input a single evaluator input $x_i \in \mathbb{F}_p$, and the coefficients $\mathbf{a}_i, \mathbf{b}_i \in \mathbb{F}_p^{S_i}$ depend on G 's input and private randomness. Let $S = \sum S_i$ denote the total number of field elements in such an encoding. *Projectivizing* such a garbling scheme reduces to *projectivizing affine maps*: if we can efficiently deliver each $\mathbf{a}_i x_i + \mathbf{b}_i$ to an evaluator who holds only bitwise labels for $\mathbf{x}_i = \text{bin}(x_i)$, then we can projectivize any affine garbling scheme.

With this context, we can appreciate why bit decomposition is so expensive: It expands each x_i into $\lg p$ field-element inputs, one per bit. Under the original scheme, each input $x_i \in \mathbb{F}_p$ requires a single affine encoding $\mathbf{a}_i x_i + \mathbf{b}_i$. But after bit decomposition, each *bit* $x_{i,j}$ requires its own affine encoding $\mathbf{a}_{i,j} x_{i,j} + \mathbf{b}_{i,j}$. Thus, encoding grows from S to at least $n \cdot S$ field elements, for a total size of Sn^2 bits.

4.3 Affine Evaluation via CRT Residues

Consider one affine map $\mathbf{a}x + \mathbf{b}$, where $x \in \mathbb{F}_p$ is E 's input and $\mathbf{a}, \mathbf{b}$ are private to G . Let x_j denote the j -th bit in a binary representation of x . Let $n = \lg p$. Projectivizing $\mathbf{a}x + \mathbf{b}$ involves computing the following:

$$\mathbf{a} \left(\sum_{j=0}^{n-1} 2^j \cdot x_j \right) + \mathbf{b}$$

We garble this map with total communication cost $O(n\lambda + nS)$ bits. The first key idea is to lift this computation from $\mathbb{F}_p$ to a CRT-friendly ring $\mathbb{Z}_M$ —a product of small prime fields—and to evaluate the affine map component-wise over each CRT residue. The main challenge is in mapping E 's input into such a representation. We next explain our approach to this mapping at a high level. Section 6 later formalizes the technique as a switch system $S_{\text{aff-}\mathbb{Z}_M}$, and Appendix C provides a concrete cost analysis.

Let $M = \prod_{i=1}^T p_i$ be the product of the first T primes, chosen so that $\lg M \geq 2n + \rho$; this ensures that arithmetic over $\mathbb{F}_p$ elements lifts to $\mathbb{Z}_M$ without wrap-around (including a masking step that we explain below). Our construction proceeds in two main stages.

Stage 1: From bits to word chunks. Our first task is to convert the bits of x into a CRT representation. Namely, we must compute $x \bmod p_i$ for each of our CRT primes p_i . The main garbled circuit trick available here is *free addition*. Suppose we choose some big enough (defined later) working modulus 2^ℓ , convert each of the bits x_j to an arithmetic representation modulo 2^ℓ , then linearly compute the following for each i :

$$\left(\sum_{j=0}^{n-1} (2^j \bmod p_i) \cdot x_j \right) \in \mathbb{Z}_{2^\ell}$$

Note that if 2^ℓ is big enough, this is congruent to $x \bmod p_j$. Computing all T such terms requires $O(Tn)$ operations, but because the computation is linear, these operations are “free” in the sense that they do not increase the size of the garbled encoding [KS08, Hea24]. The only encoding cost is from casting the bits x_j to *words* over $\mathbb{Z}_{2^\ell}$.

Unfortunately, the above strategy of casting bits one by one—even into the relatively small ring $\mathbb{Z}_{2^\ell}$ —is too expensive, as it would incur an encoding of size $O(n\ell\lambda)$, and ℓ must be $\Omega(\lg n)$ for the above computation to be correct. This would immediately be our bottleneck. Thus, as a refinement to the above, we process the bits of x in *chunks* rather than one by one. Specifically, we partition the n bits of x into $C = \lceil n/\lg n \rceil$ chunks, each consisting of $\lg n$ bits. For each chunk x_c , we use a helper switch system **bin-to-word** taken from [Hea24] (see Lemma 6.3), which *simultaneously* converts all bits of the chunk into a single arithmetic word $w_c \in \mathbb{Z}_{2^\ell}$. (Note that we cannot afford larger chunks, as the *computation* expended by G/E grows exponentially in the size of the chunks.) This amortizes the cost of conversion across the chunk. Summing over all C chunks, this stage costs $O(n\lambda)$ bits in total.

With our chunks encoded as words, we can again linearly compute values (congruent to) the correct CRT residues. Namely, for each CRT prime p_i , we compute the following:

$$r_i := \left(\sum_{c=0}^{C-1} (2^{c \lg n} \bmod p_i) \cdot w_c \right) \in \mathbb{Z}_{2^\ell}.$$

Stage 2: From CRT residues to affine encodings. For each CRT prime p_i , we now have a value $r_i \in \mathbb{Z}_{2^\ell}$ which is *congruent to*, but not equal to, $x \bmod p_i$. For each prime p_i , we apply a helper switch system **word-to-ring** (Lemma 6.4) which performs two tasks:

1. Reduces the residue r_i modulo p_i , then
2. Computes $\mathbf{a}x + \mathbf{b} \bmod p_i$ for garbler inputs $\mathbf{a}, \mathbf{b}$.

This computes the i -th component of the CRT decomposition of the affine map:

$$\mathbf{a}x + \mathbf{b} \bmod M \cong (\mathbf{a}x + \mathbf{b} \bmod p_1, \dots, \mathbf{a}x + \mathbf{b} \bmod p_T)$$

After computing all such residues, E can locally apply the CRT isomorphism to recover $\mathbf{a}x + \mathbf{b} \bmod M$.

Each instance of **word-to-ring** costs $O(S \lg n)$ bits; summing over $T = \Theta(n/\lg n)$ primes gives $O(Sn)$ bits total. Notably, this S -dependent cost carries no factor of λ : the output wires carrying the affine encodings are terminal — they are never fed into further switch gates as control inputs, and therefore they do not require the λ -fold label repetition needed for secure evaluation of our hash function. We elaborate on this distinction between wires with λ blow-up and those without in Section 6. Formalizing and applying this compatibility between arithmetic switch systems and low-overhead IT-GS-style labels is a main contribution of our approach.

Stage 3: Reducing Modulo p . Now that E holds $\mathbf{a}x + \mathbf{b} \in \mathbb{Z}_M^S$, she can simply locally compute modular reduction to obtain $\mathbf{a}x + \mathbf{b} \bmod p$. Because we set M large enough, this is indeed the correct output. However, as we have written so far, the approach is insecure:

After CRT reconstruction, E holds $\mathbf{a}x + \mathbf{b}$ as integers over $\mathbb{Z}_M$, which may reveal more about $(\mathbf{a}, \mathbf{b})$ than the same output simplified modulo p . To fix this, G adds a statistical mask $\boldsymbol{\mu} \cdot p$ to $\mathbf{b}$ when lifting to $\mathbb{Z}_M$, where $\boldsymbol{\mu}$ is chosen with enough entropy to smudge the integer quotient (see Lemma 3.2). Adding this random mask does not break correctness, because the mask vanishes modulo p . Thus, E still recovers the correct result in $\mathbb{F}_p^S$ after a final reduction, but now she learns negligible information from $\mathbf{a}x + \mathbf{b} \bmod \mathbb{Z}_M$. See Theorem 5.2 for a formal treatment of this smudging.

4.4 Application to VOLE

Our main technique—which delivers $\mathbf{a}x + \mathbf{b} \bmod p$ to E—can be understood as a reduction from chosen-input VOLE to 1-out-of-2 OT. The garbler G plays the VOLE sender, and the evaluator E the VOLE receiver.

Very specifically, we use the OT functionality to deliver E’s input labels, G sends a garbled program, and then E locally computes the output. The resulting protocol is trivially maliciously-secure against the receiver. See Section 7.3 for a formalism.

As discussed in Section 1, this is surprisingly efficient. The protocol consumes only $\lg p$ OTs and $O((\lambda + S) \lg p)$ extra bits to compute a length- S VOLE correlation over $\mathbb{F}_p$. To our knowledge, this is the best reduction from large-field VOLE to OT in Minicrypt.

4.5 Applications to Bitcoin Bridges

The projectivization recipe described above and formalized in Theorem 5.3 applies to any partial garbling scheme with component-wise affine input encodings over $\mathbb{F}_p$. We illustrate the utility of our approach with two concrete applications to verifying proofs on Bitcoin, arising from recent work [EL26, GKS⁺26].

Argo MAC encoding. Argo MACs [EL26] are homomorphic MACs over elliptic curve groups, used to efficiently authenticate group operations in garbled computations. Computing an Argo MAC on an elliptic curve point $P \in E(\mathbb{F}_p)$ amounts to evaluating $\kappa = O(\lambda)$ low-degree polynomials on the affine coordinates of P . These low-degree polynomials can be securely computed by efficient information-theoretic affine garbling techniques [IW14]. The overall function has $|x| = 2$ field inputs (the coordinates of P); the garbling of this function results in an overall encoding of size $S = O(\lambda)$ field elements, which is surprisingly small.

However, recall that for the intended use case, we need that E’s input x is encoded as a Lamport signature, and this requires the garbling scheme to be projective. [EL26] projectivizes via bit decomposition at a cost of $O(Sn^2) = O(\lambda^3)$ bits, where $n := \lg p = O(\lambda)$. Applying our technique reduces this to $O(Sn + |x|\lambda n) = O(\lambda^2)$ bits. Concretely, we reduce the size of garbling Argo MACs (for the BN254 elliptic curve group) from ≈ 6.8 MiB to ≈ 355 KiB, a $20\times$ improvement. See Section 7.1 for more details.

Private scalar multiplication. BABE [GKS⁺26] constructs a garbling scheme for scalar multiplication $(r, \pi) \mapsto r \cdot \pi$ over a BN curve, where G holds a private scalar r and E holds a curve point π . By combining this primitive with a witness encryption scheme for linear pairing relations, they achieve a mechanism for verifying a Groth16 proof on Bitcoin.

BABE reduces scalar multiplication in $\mathbb{G}$ to $n = \lg |\mathbb{G}| = O(\lambda)$ conditional point additions $\pi \mapsto r_i \cdot \pi + \phi_i$, where ϕ_i is a uniform mask chosen by G. Each of these operations is expressible as degree-2 polynomial evaluations jointly computing a function $\mathbb{F}_p^2 \rightarrow \mathbb{F}_p^3$ on the affine coordinates of π , where the coefficients of the polynomials are secrets chosen by G. The IT-GS computing this once again outputs $S = O(\lambda)$ encoding field elements. Like Argo, BABE projectivizes the above IT-GS via bit decomposition for a total communication cost of $O(\lambda^3)$ bits. Applying Theorem 5.3 improves this to $O(\lambda^2)$ bits. Concretely, we reduce the size of garbled encodings from ≈ 22 MiB to ≈ 500 KiB, a $45\times$ improvement. We present the full construction (with necessary input checks), which we call *Embryo*, in Section 7.2.

5 Projectivizing Garbling Schemes

In this section, we build up to our main result: a recipe for projectivizing any *affine* partial garbling scheme for a function whose (non-private) inputs are elements of a prime field $\mathbb{F}_p$.

5.1 Projective Partial Garbling Scheme for Affine Maps over $\mathbb{F}_p$

We begin by stating our most involved result: a *projective* garbling scheme for computing affine functions over CRT-friendly rings. The construction is formalized as a switch system [Hea24], and we defer its presentation to Section 6. Our remaining results in this section are straightforward applications of this theorem.

Theorem 5.1 (Projective Garbling Scheme for Affine Map over $\mathbb{Z}_M$). *Let $l: \{0, 1\}^n \times (\mathbb{Z}_M^S \times \mathbb{Z}_M^S) \rightarrow \mathbb{Z}_M^S$ be defined as*

$$l(\mathbf{x}, \mathbf{a}, \mathbf{b}) := \mathbf{a} \cdot \left(\sum_{j=0}^{n-1} 2^j x_j \right) + \mathbf{b} \bmod M$$

where M is a primorial larger than 2^n . Assuming the existence of a CCRH, there exists a projective partial garbling scheme $\text{PGS}_{\text{aff-}\mathbb{Z}_M}$ with garbled program size $O((\lambda + S) \lg M)$ bits.

Proof. Deferred to Section 6. We construct a switch system $S_{\text{aff-}\mathbb{Z}_M}$ computing l and invoke Theorem 6.1. $\square$

Lifting Affine Maps from $\mathbb{F}_p$ to $\mathbb{Z}_M$. Given a partial garbling scheme for affine maps over $\mathbb{Z}_M$ for a primorial M , we describe how to construct a partial garbling scheme for affine maps over $\mathbb{F}_p$. Throughout, we identify elements of $\mathbb{F}_p$ with their integer representatives in $\{0, \dots, p-1\}$ and perform arithmetic over $\mathbb{Z}$.

The product $a \cdot x$ lies in $[0, (p-1)^2]$ and adding b gives a value in $[0, p^2)$. Hence, if $M \geq p^2$, no reduction modulo M occurs and

$$(a \cdot x + b) \bmod p = ax + b \in \mathbb{F}_p.$$

In our construction, the garbler adds a statistical mask $\mu \cdot p$ (with μ sampled uniformly from a large domain) that vanishes modulo p but hides the integer quotient $\lfloor (a \cdot x + b)/p \rfloor$, which would otherwise leak the size of a, b . To accommodate both the product and the mask without overflow, we require

$$m := \lg M \geq 2n + \rho,$$

where $n = \lg p$ and ρ is a statistical security parameter.

Theorem 5.2 (Projective Garbling Scheme for Affine Map over $\mathbb{F}_p$). *Let $l: \{0, 1\}^n \times (\mathbb{F}_p^S \times \mathbb{F}_p^S) \rightarrow \mathbb{F}_p^S$ be defined as*

$$l(\mathbf{x}, \mathbf{a}, \mathbf{b}) = \mathbf{a} \cdot \left(\sum_{j=0}^{n-1} 2^j x_j \right) + \mathbf{b} \bmod p$$

where p is a prime modulus and $n = \lg p$. Assuming the existence of a CCRH, there exists a projective partial garbling scheme $\text{PGS}_{\text{aff-}\mathbb{F}_p}$ with garbled program size $O(n(\lambda + S))$ bits.

Proof. We construct $\text{PGS}_{\text{aff-}\mathbb{F}_p} = (\text{Garb}, \text{Enc}, \text{Eval})$ as follows. Let M be the first primorial larger than $p^2 \cdot 2^\rho$ i.e. $M = \prod_{t=1}^T p_t$ is the product of the first T primes, with T chosen minimally such that $M \geq p^2 \cdot 2^\rho$. Let $m := \lg M$. Observe that $m = O(n)$ (we assume $\rho < n$). By the prime number theorem, $p_T = \Theta(n)$, $T = \Theta(n/\lg n)$.

1. $\text{Garb}(1^\lambda, \mathbf{a}, \mathbf{b})$: Interpret $\mathbf{a}, \mathbf{b}$ over $\mathbb{Z}_M$. Sample $\mu \in [(M - p^2)/p]^S$ uniformly at random, define $\mathbf{b}' := \mathbf{b} + \mu \cdot p$. Output $(\tilde{f}, \tilde{e}) = \text{Garb}_{\text{aff-}\mathbb{Z}_M}(1^\lambda, \mathbf{a}, \mathbf{b}')$.
2. $\text{Enc}(e, \mathbf{x})$: Output $\tilde{\mathbf{x}} = \text{Enc}_{\text{aff-}\mathbb{Z}_M}(e, \mathbf{x})$.
3. $\text{Eval}(\tilde{f}, \tilde{\mathbf{x}})$: Compute $\mathbf{z}' = \text{Eval}(\tilde{f}, \tilde{\mathbf{x}})$, reduce it via $\mathbf{z} := \mathbf{z}' \bmod p$, and output $\mathbf{z} \in \mathbb{F}_p^S$.

Correctness follows immediately after observing there is no wrap-around modulo M . For privacy, note that μ is sampled with enough entropy such that $\mathbf{z}'$ can be simulated from the output $\mathbf{z} \in \mathbb{F}_p^S$ by independently sampling $\mu \in [(M - p^2)/p]^S$ and computing $\mathbf{z}' = \mathbf{z} + \mu \cdot p$. This is statistically close to the real distribution of $\mathbf{z}'$ via the Smudging Lemma (Lemma 3.2). $\square$

5.2 From Affine Encodings to Projective Schemes

With a projective garbling scheme for individual affine maps over $\mathbb{F}_p$ in hand, we now show how to use it as a generic transformation: any partial garbling scheme whose input encoding is component-wise affine over $\mathbb{F}_p$ can be upgraded to a projective scheme at a modest additive cost. The idea is simple—apply $\text{PGS}_{\text{aff-}\mathbb{F}_p}$ independently to each input component, converting every affine encoding $x_i \mapsto \mathbf{a}_i \cdot x_i + \mathbf{b}_i$ into a projective one that operates on the bits of x_i . The evaluator first recovers the original input labels from the bit-level garbled inputs via $\text{Eval}_{\text{aff-}\mathbb{F}_p}$, then proceeds with the original scheme's evaluation.

Theorem 5.3 (Projectivizing Affine Garbling Schemes). *Let $f: \mathcal{X} \times \mathcal{Y} \rightarrow \mathcal{Z}$ be a function, where $\mathcal{X} = \mathbb{F}_p^{|x|}$. Suppose there exists an affine partial garbling scheme $\text{PGS} = (\text{Garb}, \text{Enc}, \text{Eval})$ for f . Specifically, assume*

1. Garb outputs $(\tilde{f}, e)$, where $e = (\mathbf{a}_i, \mathbf{b}_i)_{i \in [|x|]}$ for $\mathbf{a}_i, \mathbf{b}_i \in \mathbb{F}_p^{S_i}$.

$$2. \text{Enc}(e, x) := (\mathbf{a}_i \cdot x_i + \mathbf{b}_i)_{i \in [|x|]}.$$

Set $S := \sum_{i \in [|x|]} S_i$. Define the projective function

$$f': \{0, 1\}^{n|x|} \times \mathcal{Y} \rightarrow \mathcal{Z}$$

where $n := \lg p$ by first recombining bits of x to field elements via $\text{bin-to-}\mathbb{F}_p$ and then computing f . Then, assuming the existence of CCRH, there exists a projective partial garbling scheme PGS' for f' with total communication $|\tilde{f}| + O(n(|x|\lambda + S))$ bits.

Proof. We construct $\text{PGS}' = (\text{Garb}', \text{Enc}', \text{Eval}')$ as follows:

1. $\text{Garb}'(1^\lambda, y)$: Compute $(\tilde{f}, e) := \text{Garb}(1^\lambda, y)$ to obtain $e = (\mathbf{a}_i, \mathbf{b}_i)_{i \in [|x|]}$, which takes this form by assumption. For each use the affine map garbling scheme from Theorem 5.2 to compute

$$(\tilde{f}_i, \tilde{e}_i) := \text{Garb}_{\text{aff-}\mathbb{F}_p}(1^\lambda, (\mathbf{a}_i, \mathbf{b}_i)).$$

Output garbled program $(\tilde{f}, \tilde{f}_1, \dots, \tilde{f}_{|x|})$ and encoding information $(\tilde{e}_1, \dots, \tilde{e}_{|x|})$.

2. $\text{Enc}'((\tilde{e}_1, \dots, \tilde{e}_{|x|}), \mathbf{x})$: Parse $\mathbf{x} \in \{0, 1\}^{n|x|}$ as $(\mathbf{x}_1, \dots, \mathbf{x}_{|x|})$ where each $\mathbf{x}_i \in \{0, 1\}^n$. For each $i \in [|x|]$, compute $\tilde{\mathbf{x}}_i := \text{Enc}_{\text{aff-}\mathbb{F}_p}(\tilde{e}_i, \mathbf{x}_i)$. Output $(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_{|x|})$.
3. $\text{Eval}'((\tilde{f}, \tilde{f}_1, \dots, \tilde{f}_{|x|}), (\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_{|x|}))$: For each $i \in [|x|]$, compute $\mathbf{c}_i := \text{Eval}_{\text{aff-}\mathbb{F}_p}(\tilde{f}_i, \tilde{\mathbf{x}}_i)$, obtaining $\mathbf{c}_i = \mathbf{a}_i \cdot x_i + \mathbf{b}_i \in \mathbb{F}_p^{S_i}$. Then output $\text{Eval}(\tilde{f}, (\mathbf{c}_1, \dots, \mathbf{c}_{|x|}))$.

Correctness. Each $\text{Eval}_{\text{aff-}\mathbb{F}_p}$ correctly recovers $\mathbf{a}_i \cdot x_i + \mathbf{b}_i = \text{Enc}(e, x)_i$ by Theorem 5.2, so the final Eval call receives the same input encodings as in the original scheme, and correctness of PGS yields the result.

Privacy. Follows by a standard hybrid argument over $i \in [|x|]$, invoking the privacy of $\text{PGS}_{\text{aff-}\mathbb{F}_p}$ in each step and the privacy of PGS in the final step.

Communication. The garbled program consists of $\tilde{f}$ from the original scheme plus $|x|$ instances of $\text{PGS}_{\text{aff-}\mathbb{F}_p}$, each of size $O(S_i n + \lambda n)$ by Theorem 5.2. Summing over i gives total communication $O(|\tilde{f}| + Sn + |x|\lambda n)$ bits, as required. $\square$

Remark 5.1. Technically, Theorem 5.3 holds even if $\mathcal{X} = \mathcal{X}_1 \times \dots \times \mathcal{X}_{|x|}$ is some subset of $\mathbb{F}_p^{|x|}$. Note that affine is still well-defined (Definition 1). In this case, the projective function f' inherits the domain restriction so if the bit-decomposed input $\mathbf{x} \in \{0, 1\}^{n|x|}$ does not correspond to a valid element $x \in \mathcal{X}$, then we make no guarantees on correctness or privacy. This extension is required in our application to Argo MACs in Section 7.1.

6 Switch System for Affine Maps

In this section, we prove Theorem 5.1 by constructing a switch system $S_{\text{aff-}\mathbb{Z}_M}$ for affine maps over $\mathbb{Z}_M$, where M is a primorial (in particular, it is CRT-friendly), and then invoking Theorem 6.1, which shows how to garble a switch system and states associated costs.

6.1 Switch Systems with IT Labels

Our construction of $S_{\text{aff-}\mathbb{Z}_M}$ builds on the switch system framework of [Hea24], which provides a general language for expressing garbled computations over integer rings. The original framework was designed to suit the needs of arithmetic garbling, for which it sufficed to consider wires over rings of the form $\mathbb{Z}_{2^k}$. We observe that the framework generalizes naturally to wires over arbitrary finite groups. We also introduce a distinction between two wire types: *control-friendly* wires, which serve as inputs to switch gates and therefore require labels with λ -fold repetition to ensure hash security, and plain wires, which merely carry data and admit short one-time-pad labels. This distinction is important: it is precisely what allows IT-GS labels, which are single field elements rather than λ -bit strings, to flow through the system without incurring the λ blowup that would otherwise dominate communication.

Definition 3 (IT-Friendly Switch Systems). A **switch system** is a system of constraints on wires (i.e. constrained variables), where each wire w_i holds a value over some finite group $\mathbb{G}_i$. The system is defined in terms of gates. Non-input wires are initially not set (i.e., have no value), and as the system runs, wires become set according to the rules of each gate. We classify wires into two types – those that are **control-friendly**, and those that are not. Each control-friendly wire has a value in a ring expressible as the direct product of integer rings $\mathbb{Z}_{2^{k_0}} \times \mathbb{Z}_{2^{k_1}} \times \dots$ for wire-specific k_i ; non-control-friendly wires can be over any finite group. The types of gates are as follows:

- A **switch** takes as input a control-friendly wire s over $\mathbb{Z}_2$, as well as a wire x (which may or may not be control friendly) over some finite group $\mathbb{G}$. It outputs data wire $y \in \mathbb{G}$. y is control-friendly iff x is. We refer to the wire s as a control wire. If s becomes set and has value 0, then the switch establishes constraint $x = y$. We denote a switch by writing:

$$y := x \Leftarrow \text{ctrl} \stackrel{?}{=} 0$$

- A **join** takes input wires x, y from some finite group $\mathbb{G}$, where either both wires are control-friendly, or neither are. It establishes constraint $x = y$. We denote such a gate by simply writing $x = y$.
- A **pair** gate takes input wires $x \in \mathbb{G}_0$ and $y \in \mathbb{G}_1$, where either both wires are control-friendly, or neither are. It outputs wire $z \in \mathbb{G}_0 \times \mathbb{G}_1$ and enforces that $z = (x, y)$. z is control-friendly iff x, y are. We denote such a gate by writing:

$$z := (x, y).$$

- A **homomorphism** gate is parameterized by a group homomorphism $\varphi : \mathbb{G}_0 \rightarrow \mathbb{G}_1$. It takes as input a wire $x \in \mathbb{G}_0$. The gate outputs $y \in \mathbb{G}_1$. It establishes the constraint $y = \varphi(x)$. y is control-friendly iff x is. We denote such a gate by writing $y := \varphi(x)$.
- A **subgroup** gate takes as input a wire $x \in \mathbb{G}_0$ where it can be statically deduced from the switch system structure that x is, in fact, in $\mathbb{G}_1$, a subgroup of $\mathbb{G}_0$. The gate outputs a wire $y \in \mathbb{G}_1$. y is control-friendly iff x is. For compatibility with garbled labels, the gate is parameterized by a function $\varphi : \mathbb{G}_0 \rightarrow \mathbb{G}_1$ such that the following “affine” property holds:

$$\varphi(x + y) = \varphi(x) + y \quad \text{for any } x \in \mathbb{G}_0, y \in \mathbb{G}_1$$

If wire x becomes set, the gate sets y to $x \in \mathbb{G}_1$. We denote such a gate by writing $y := \varphi(x)$.

A switch system may have constant wires, which hold some arbitrary public constant from an arbitrary group. Constant wires may be control-friendly or non-control-friendly.

For convenience, we assume each gate has a unique identifier gid . A switch system has two distinguished input wires, one control-friendly wire from ring R_{in} , and one non-control-friendly from group $\mathbb{G}_{in}$. It has a distinguished non-control-friendly output wire from group $\mathbb{G}_{out}$. We say a system is **legal** if for any input $x \in R_{in} \times \mathbb{G}_{in}$ there exists a unique assignment of each wire to a value that satisfies all gate constraints. Because assignments are unique, a legal switch system P computes a function $P : R_{in} \times \mathbb{G}_{in} \rightarrow \mathbb{G}_{out}$. From here on, we only consider legal systems.

Generalizing [Hea24]. Definition 3 is different from the switch system presentation of [Hea24] in a few notable ways:

- The above definition adds a distinction between *control-friendly* and *non-control-friendly* wires. This distinction is needed in our setting to demonstrate that switch systems can correctly interface with IT-GS-style labels. It is precisely this generalization that enables our approach. In [Hea24], all wires are implicitly control-friendly.
- [Hea24] provides gates that perform specific operations over rings of form $\mathbb{Z}_{2^k}$, including gates that compute affine operations, gates that compute modular reduction, and gates that perform a restricted form of division. Because our formalism must operate over a broader family of sets, we generalize these notions to *homomorphism* and *subgroup* gates.

We argue that our presentation is, in fact, a strict generalization, and therefore any switch system written in the [Hea24] formalism is automatically a switch system in ours (over control-friendly wires) as well.

First, [Hea24]’s affine gates can be handled by homomorphisms. Indeed, it is a straightforward fact that the map $(x, y) \mapsto x + y : \mathbb{G}^2 \rightarrow \mathbb{G}$ is a homomorphism for any commutative group $\mathbb{G}$. Hence, to formalize an addition gate under our definition, one need only use a pair gate to gather x, y into a pair, then use a homomorphism gate to add them together. We can formalize subtraction gates and multiplication-by-constant gates in the same manner. Affine shifts are captured by utilizing constant-valued wires.

Second, [Hea24] provides a *modular reduction* gate which takes as input a wire $x \in \mathbb{Z}_{2^{k+c}}$ and outputs $x \bmod \mathbb{Z}_{2^k}$. This gate is trivially captured as a homomorphism gate.

Finally, [Hea24] provides a *division* gate, which takes as input some wire $x \in \mathbb{Z}_{2^{k+c}}$ where it can be statically deduced from the circuit structure that x is *necessarily* a multiple of 2^c . When this is statically guaranteed, one can include a gate that computes $x/2^c \in \mathbb{Z}_{2^k}$. In our formalism, this is captured by a subgroup gate, followed by a homomorphism gate. Indeed, the elements $x = 2^c y$ clearly form a size 2^k subgroup of $\mathbb{Z}_{2^{k+c}}$. The most complex component of Definition 3 is that for such subgroup gates, one must specify some map φ that respects the structure of the subgroup; this map is needed to garble the system. In the case of division, this map φ is one that takes an element of $\mathbb{Z}_{2^{k+c}}$ and sets its low c bits to all-zeros. For those elements in the subgroup, φ is indeed a no-op, and moreover, we do have $\varphi(a+b) = \varphi(a) + b$ for element a in $\mathbb{Z}_{2^{k+c}}$ and b in the subgroup. From here the map that sends $2^c y$ to y is trivially a homomorphism from the subgroup to $\mathbb{Z}_{2^k}$.

Hence, our formalism captures all gates used in the definition of [Hea24], so we can use their switch system constructions.

We note that [Hea24] defined a notion of *oblivious* switch systems, i.e. a switch system with randomized inputs where all switch controls can be simulated. There is no difficulty in adding such a notion to Definition 3; but we do not need it for our setting. In our setting, all switch controls will be dictated by E ’s input x , so already all switch controls can be simulated trivially.

Switch system cost. The communication cost of garbling a switch system is captured by its *join width*. Intuitively, join gates are the only gates that contribute to program size, since all other gate types allow the evaluator to derive output labels for free.

Definition 4 (Join Width). *Consider a join gate constraining $x = y$ where variables x, y are over a group $\mathbb{G}$. We say that this gate $x = y$ has a **join width** as follows:*

$$\text{join-width}(x = y) = \begin{cases} \lambda \lg |\mathbb{G}| & \text{if } x, y \text{ are control-friendly} \\ \lg |\mathbb{G}| & \text{otherwise} \end{cases}$$

For a switch system S , the join width of S is defined as the sum of the join widths of each of its join gates.

We now state the main result of this subsection: any legal switch system in the above framework admits a projective partial garbling scheme whose program size is exactly its join width. With this theorem, we reduce the task of constructing a projective partial garbling scheme for affine maps to constructing a switch system for the same.

In short, the following construction largely follows that of [Hea24]. Every wire is encoded by some garbled label. Switches are handled by invoking a hash function that allows/disallows E from learning the difference between labels on the two data wires, joins are handled by explicitly including in the garbled program the difference between labels, and other gates types are handled homomorphically.

Theorem 6.1. *Let S be a switch system:*

$$S : R_{in} \times \mathbb{G}_{in} \rightarrow \mathbb{G}_{out}$$

Let S have join width w . Assuming a CCRH, there exists a partial garbling scheme for any such system with the following efficiencies:

- Program encodings are of size $w + |\mathbb{G}_{out}|$ bits.
- The input encoding string e consists of λ elements from R_{in} .

Proof. By construction of a garbling scheme (**Garb**, **Enc**, **Eval**). We describe these procedures below. Note that our scheme is a natural generalization of that of [Hea24], extended to support short, information-theoretic-secure garbled labels.

As is typical in many GC techniques, both **Garb** and **Eval** will proceed gate-by-gate. Our scheme arranges the following:

- Recall that each control-friendly wire is over some wire-specific ring R which is a product of integer rings. For each such control-friendly wire $x \in R$, the evaluator will ultimately obtain a value of form $X + x\Delta_R$ on each control-friendly wire x . Here, X is a one-time-pad mask, and Δ_R is some global correlation, akin to a Free XOR correlation [KS08].
- For each non-control-friendly wire $x \in \mathbb{G}$, the evaluator will ultimately obtain a value of form $X + x$, where X is a one-time-pad mask.

Garb first samples a random Free-XOR-style correlation:

$$\Delta \in_{\$} \{0, 1\}^\lambda$$

Note that we can interpret Δ as being written over the 0 and 1 elements of *any* ring R ; we write $\Delta_R \in R^\lambda$ to denote this.

Next, **Garb** uniformly samples a one-time-pad mask for the input evaluator's input wire $x \in R_{in}$:

$$X \in_{\$} R_{in}^\lambda$$

Our scheme's input encoding strings e are simply (Δ, X) , and the **Enc** procedure is the following affine map:

$$\text{Enc}((\Delta, X), x) = X + x \cdot \Delta_{R_{in}}$$

Recall that the program also takes as input the garbler's input $y \in \mathbb{G}_{in}$, so we need a mask for this as well. The garbling scheme simply sets the corresponding mask Y as $-y$; this straightforwardly arranges that the evaluator holds $Y + y = 0 \in \mathbb{G}_{in}$, without any communication. This same approach is used to handle constants.

We next describe how each gate type is handled by both **Garb** and **Eval**.

- **Switches.** Consider a switch $y := x \Leftarrow s = 0$. Note that by definition, the wire s is control-friendly and is over the ring $\mathbb{Z}_2$. Thus, it has a mask $S \in \mathbb{Z}_2^\lambda$, and at runtime the evaluator will hold $S + s\Delta \in \mathbb{Z}_2^\lambda$. Let x have mask X . **Garb** computes a mask on Y as follows:

$$Y := H(S, \text{gid}) + X$$

See discussion of H in Section 3.2. Here gid is the gate's unique identifier. Notice that if $s = 0$, then the evaluator holds S , and hence she can call $H(S, \text{gid})$, obtaining the value $Y - X$, allowing her to translate between values on the x and y wires; that is, if $s = 0$, the evaluator can indeed enforce the constraint $x = y$.

- **Joins.** Consider a join $x = y$. Let X, Y be the respective one-time-pad masks on these wires. The garbler appends value $X - Y$ to the program encoding $\tilde{f}$. At runtime, this allows the evaluator to enforce constraint $x = y$.
- **Pair Gates.** Consider a pair gate $z := (x, y)$, and let X, Y be the respective one-time-pad masks on the input wires. The garbler simply defines output wire mask Z as $Z = (X, Y)$; this trivially allows the evaluator to enforce constraint $z = (x, y)$.
- **Homomorphism Gates.** Consider a homomorphism gate $y := \varphi(x)$. Let X be x 's wire mask.
 - If x is not control-friendly, the garbler simply defines $Y := \varphi(X)$. Note that because labels are additive, φ is homomorphic over the evaluator's label:

$$\varphi(X + x) = \varphi(X) + \varphi(x) = Y + y$$

Thus, the evaluator can enforce constraint $y = \varphi(x)$ by simply applying φ to garbled labels.

- If x is control-friendly, we extend $\varphi : R \rightarrow S$ to $\varphi : R^\lambda \rightarrow S^\lambda$ in the natural coordinate-wise manner, and G sets $Y := \varphi(X)$. Similar to above, the evaluator enforces constraint $y = \varphi(x)$ by applying φ to her labels:

$$\varphi(X + x\Delta_R) = \varphi(X) + \varphi(x)\varphi(\Delta_R) = Y + y\Delta_S$$

- **Subgroup Gates.** Consider a subgroup gate $y := \varphi(x)$. Recall that using this gate requires that it be statically deducible that $x \in \mathbb{G}_0$ is, in fact, in some subgroup $\mathbb{G}_1$ which is the image of φ ; ultimately, the gate simply sets $y = x$, but y is in the subgroup. Let X be x 's wire mask.

- If x is not control-friendly, the garbler simply defines $Y := \varphi(X)$. Recall that φ satisfies an “affine” property over the subgroup. Thus, φ applies the correct transformation to the evaluator's label:

$$\varphi(X + x \in \mathbb{G}_0) = \varphi(X) + x = Y + y \in \mathbb{G}_1$$

- If x is control-friendly, we extend $\varphi : \mathbb{G}_0 \rightarrow \mathbb{G}_1$ to $\varphi : \mathbb{G}_0^\lambda \rightarrow \mathbb{G}_1^\lambda$ in the natural coordinate-wise manner, and G sets $Y := \varphi(X)$. E similarly applies φ to her label:

$$\varphi(X + x\Delta_{\mathbb{G}_0}) = \varphi(X) + x\Delta_{\mathbb{G}_1} = Y + y\Delta_{\mathbb{G}_1}$$

Now, recall there is a single output wire $S(x, y) \in \mathbb{G}_{out}$. By the above construction, the evaluator will at runtime hold $S(x, y) + X$ where X is some one-time-pad mask. Thus, the garbler appends X to the program encoding $\tilde{f}$ such that the evaluator can subtract it off and obtain the cleartext output $S(x, y)$.

Simulation. We now show the above scheme achieves privacy by constructing a simulator $\mathcal{S}$. Recall that our goal is to demonstrate the following indistinguishability:

$$\left\{ (\tilde{f}, \tilde{x}) \mid \begin{array}{l} (\tilde{f}, e) := \text{Garb}(1^\lambda, y) \\ \tilde{x} := \text{Enc}(e, x) \end{array} \right\} \approx \left\{ \mathcal{S}(1^\lambda, x, f(x, y)) \right\}$$

The simulator proceeds as follows. First, it samples an input encoding $\tilde{x}$ uniformly at random from R_{in} . This is a good simulation because in the real garbling, the input encoding is masked by a one-time pad. From here, the simulator proceeds as the garbler would for each gate type, using input wire gate labels to derive output labels. Indeed the handling is identical to **Garb** for each gate type except the switch.

Consider a switch of form $y := x \leftarrow s = 0$. By induction, suppose the simulator has already chosen labels S, X respectively for wires s, x . The simulator then proceeds conditionally:

- If the switch has control 0, the simulator's behavior matches the garbler's:

$$Y := H(S, gid) + X$$

This simulation is consistent, since in the real world evaluation, the evaluator can evaluate the hash function herself.

- If the switch has control 1, the simulator instead samples Y uniformly from the appropriate group.

Choosing output labels from switches uniformly at random is a good simulation due to properties of H . In particular, recall from Section 3.2's discussion that we use H in two ways:

- For control-friendly data wires, we use the *circular* correlation-robustness of H to encrypt a value that involves Δ .
- For *non*-control-friendly data wires, we use the (non-circular) correlation-robustness of H to generate a uniform group element.

In the real garbling, all switches with control 1 take the following form, from the perspective of the evaluator:

$$\begin{cases} Y := H(S \oplus \Delta, gid) + X + (y - x)\Delta & \text{if data wires are control friendly} \\ Y := H(S \oplus \Delta, gid) + X + (y - x) & \text{otherwise} \end{cases}$$

That is, the hashed value differs from the evaluator's label S by the correlation Δ , and the labels X and Y differ by an amount related to their cleartext values. This is the *only* place where Δ appears in the evaluator's view. All other labels/ciphertexts are simply homomorphically derived from labels in $\tilde{x}$ and join gate material included in $\tilde{f}$. Therefore, we can externalize the calls to H that involve Δ to an oracle, then apply the CCRH security property to replace these calls to H by a uniform function. Thus, simulating each label on the output of a switch via a uniform encoding is indeed indistinguishable from the real garbling. $\square$

6.2 Helper Switch Systems

With the appropriate definitions in place, we now introduce some helper switch systems that serve as building blocks for $S_{\text{aff-}\mathbb{Z}_M}$. Each is stated with its join width, which will be tallied in Section 6.3 to establish the overall cost of our construction. We also recall two simple lemmas about one-hot encodings that the constructions rely on.

Our helper systems make extensive use of *one-hot encodings*, which enables performing arbitrary function evaluation inside a switch system.

Lemma 6.1 (One-Hot Truth Table Evaluation). *Let R denote an arbitrary ring. Let $x \in [n]$ be a value, and let $h = \text{hot}(x) \in R^n$ denote a one-hot encoding of x , written over R . Let $f : [n] \rightarrow R$ denote an arbitrary function, and let $\mathcal{T}(f) \in R^n$ denote its truth table. The following holds:*

$$\left(\sum_{i=1}^n \mathcal{T}(f)_i \cdot h_i \right) = f(x)$$

Proof. Since $h = \text{hot}(x)$, we have $h_i = 1$ if $i = x$ and $h_i = 0$ otherwise, so

$$\sum_{i=1}^n \mathcal{T}(f)_i \cdot h_i = \mathcal{T}(f)_x = f(x). \quad \square$$

We now present the helper switch systems. These are straight-forward modifications of macros in [Hea24]. The first one **scale-hot** scales a length- n (control-friendly) one-hot encoding by an arbitrary group element, which may or may not be control-friendly.

Lemma 6.2 (scale-hot). *Let $\mathbb{G}$ be a finite group and let $n \geq 1$. There exists a switch system computing the following function, given that h is a one-hot encoding:*

$$\begin{aligned} \text{scale-hot} : \{0, 1\}^n \times \mathbb{G} &\rightarrow \mathbb{G}^n \\ \text{scale-hot}(h, s)_i &= \begin{cases} 0 \in \mathbb{G} & \text{if } h_i = 0 \\ s & \text{otherwise} \end{cases} \end{aligned}$$

The system has n switches and join-width $\lambda \lg |\mathbb{G}|$ if s is control-friendly, and $\lg |\mathbb{G}|$ otherwise.

Proof. We generate the following constraints for each $i \in [n]$:

$$y_i := (0 \in \mathbb{G}) \Leftarrow h_i = 0 \quad \sum_i y_i = s$$

where y is the output vector. If $h_\alpha = 1$, then all $y_{i \neq \alpha} = 0$, and hence $y_\alpha = s$ by the join constraint, correctly implementing the desired function. The join constraint $\sum_i y_i = s$ contributes $\lambda \lg |\mathbb{G}|$ bits if s is control-friendly and $\lg |\mathbb{G}|$ otherwise; the switch constraints contribute nothing to join width. $\square$

Next, we introduce a switch system which converts bits of a short integer to an arithmetic word. We require that the integer be *short* as we use one-hot encodings as a stepping stone.

Lemma 6.3 (bin-to-word). *Let $n, k \geq 1$. There exists a switch system (with all wires control-friendly) computing the following function:*

$$\begin{aligned} \text{bin-to-word} &: \{0, 1\}^n \rightarrow \mathbb{Z}_{2^k} \\ \text{bin-to-word}(x) &= \text{bin}^{-1}(x) \bmod 2^k \end{aligned}$$

The system has join-width $(n - 1 + k)\lambda$.

Proof. We build on the following switch system from [Hea24], which converts a binary string to a one-hot encoding with all wires control-friendly:

$$\begin{aligned} \text{bin-to-hot} &: \{0, 1\}^n \rightarrow \{0, 1\}^{2^n} \\ \text{bin-to-hot}(x)_i &= \begin{cases} 1 & \text{if } i = x \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

This system has join-width $(n - 1)\lambda$ and $O(2^n)$ switches. Using `bin-to-hot`, `scale-hot`, and Lemma 6.1, we define:

$$\text{bin-to-word}(c) = \sum_i \mathcal{T}(\text{id})_i \cdot (\text{scale-hot}(\text{bin-to-hot}(c), 1 \in \mathbb{Z}_{2^k}))_i$$

where $\text{id}(x) = x \in \mathbb{Z}_{2^k}$. Correctness is immediate from Lemma 6.1. The constant $1 \in \mathbb{Z}_{2^k}$ is control-friendly, so `scale-hot` contributes $k\lambda$ join-width by Lemma 6.2, giving total join-width $(n - 1 + k)\lambda$. Note that this construction is implicit in [Hea24], except we do not need one-time masks here. $\square$

Our third helper addresses the following task: given a word encoding (over a control-friendly wire), evaluate an arbitrary function g , scale it by a value $a \in R$ and add offset $b \in R$.

Lemma 6.4 (word-to-ring). *Let R be a finite ring, let $k \geq 1$, and let $g : \mathbb{Z}_{2^k} \rightarrow R$ be an arbitrary function. There exists a switch system computing the following function:*

$$\begin{aligned} \text{word-to-ring}_g &: \mathbb{Z}_{2^k} \times (R \times R) \rightarrow R \\ \text{word-to-ring}_g(x, (a, b)) &= a \cdot g(x) + b \end{aligned}$$

The system has join-width $2(k - 1)\lambda + \lg |R|$.

Proof. We make use of the following helper function and switch system from [Hea24]:

$$\begin{aligned} \text{word-to-bin} &: \mathbb{Z}_{2^k} \rightarrow \{0, 1\}^k, \quad \text{word-to-bin}(x) = y \text{ where } \sum_i 2^i y_i = x \\ \text{word-to-hot} &: \mathbb{Z}_{2^k} \rightarrow \{0, 1\}^{2^k}, \quad \text{word-to-hot} = \text{bin-to-hot} \circ \text{word-to-bin} \end{aligned}$$

The system `word-to-hot` has join-width $2(k - 1)\lambda$ under Definition 4 and $O(2^k)$ switches, each with $O(k)$ -bit data wires.

To compute `word-to-ringg`, we observe that it factors as:

$$\text{word-to-ring}_g(x, (a, b)) = \text{hot-to-ring}_g(\text{word-to-hot}(x), (a, b))$$

where `hot-to-ringg` evaluates g on the index of the hot entry via Lemma 6.1 and scales by a . The join contributed by `hot-to-ringg` is $\lg |R|$ (a single non-control-friendly join enforcing $\sum_i y_i = a$). The total join width is therefore $2(k - 1)\lambda + \lg |R|$. $\square$

Remark 6.1 (Computational cost of `word-to-ring`). *In our application of `word-to-ring`, we instantiate k with a working modulus $\ell \approx 3 \lg n$. Each invocation of `word-to-ring` incurs $2^\ell(\ell + 1)$ hash evaluations. Summed over $T = \Theta(n / \lg n)$ primes, this gives $\Theta(n^4)$ total hash calls, a substantial cost. In Appendix A, we present a more computationally efficient but restricted variant of `word-to-hot` that suffices for our application and reduces this to $O(n^2)$ hash calls.*

6.3 Construction of $S_{\text{aff-}\mathbb{Z}_M}$

We now construct a switch system $S_{\text{aff-}\mathbb{Z}_M}$ that computes affine maps of the form $\mathbf{a}x + \mathbf{b}$ by composing the helper systems from the previous subsection.

Parameters. Recall $M = p_1 \times p_2 \times \cdots \times p_T$ where p_i is the i -th prime, $m := \lg M \geq n$. Set $C := \lceil n / \lg n \rceil$ as the number of chunks, $\ell := \lg(C \cdot n \cdot p_T) = O(\lg m)$ as a working modulus, chosen so that residue accumulation in Step 2 does not overflow $\mathbb{Z}_{2^\ell}$, and $T = O(m / \lg m)$ as the number of prime factors of M .

Step 1: Chunk conversion. Partition the input $x \in \{0, 1\}^n$ into C chunks $x_0, \dots, x_{C-1}$, each of length $\lg n$ bits. For each $c \in [C]$, apply **bin-to-word** (Lemma 6.3) to obtain a control-friendly wire $w_c \in \mathbb{Z}_{2^\ell}$ carrying $\text{bin}^{-1}(x_c) \bmod 2^\ell$.

Step 2: Free accumulation into CRT residues. For each prime $p_i \in [T]$, compute a control-friendly wire $r_i \in \mathbb{Z}_{2^\ell}$ via:

$$r_i := \sum_{c=0}^{C-1} (2^{c \lg n} \bmod p_i) \cdot w_c \in \mathbb{Z}_{2^\ell}$$

Each scalar multiplication (by public constant) and addition is a homomorphism; neither contributes join width. Correctness: since $x = \sum_c \text{bin}^{-1}(x_c) \cdot 2^{(c-1) \lg n}$, we have $r_i \equiv x \pmod{p_i}$, and our choice of ℓ ensures no overflow.

Step 3: Residue evaluation. For each prime $p_i \in [T]$, apply **word-to-ring** $_{g_i}$ (Lemma 6.4) to the wire r_i , where:

$$g_i : \mathbb{Z}_{2^\ell} \rightarrow \mathbb{Z}_{p_i}^S, \quad g_i(r) := (r \bmod p_i) \cdot \mathbf{1}_S$$

with garbler inputs $(\mathbf{a} \bmod p_i, \mathbf{b} \bmod p_i) \in \mathbb{Z}_{p_i}^S \times \mathbb{Z}_{p_i}^S$. This produces an output wire $o_i \in \mathbb{Z}_{p_i}^S$ carrying $\mathbf{a}x + \mathbf{b} \in \mathbb{Z}_{p_i}^S$.

Output. The system outputs the tuple $(o_1, \dots, o_T)$ via pair gates. By the CRT isomorphism $\mathbb{Z}_M^S \cong \prod_{i=1}^T \mathbb{Z}_{p_i}^S$, this tuple is exactly the CRT representation of $\mathbf{a} \cdot x + \mathbf{b} \in \mathbb{Z}_M^S$.

6.3.1 Join-width of $S_{\text{aff-}\mathbb{Z}_M}$

We now tally join widths gate-by-gate across the three steps.

1. Each of the C instances of **bin-to-word** is invoked with input length $\lg n$ and output ring $\mathbb{Z}_{2^\ell}$, contributing join-width $(\lg n - 1 + \ell)\lambda = O(\lg m \cdot \lambda)$ per instance by Lemma 6.3. Summing over $C = \lceil n / \lg n \rceil$ chunks gives $O(m\lambda)$ total, noting $m \geq n$.
2. Homomorphism gates contribute no join width.
3. Each of the T instances of **word-to-ring** $_{g_i}$ is invoked with $k = \ell = O(\lg m)$ and $R = \mathbb{Z}_{p_i}^S$, contributing $2(\ell - 1)\lambda + S \lg p_i = O(\lg m \cdot \lambda + S \lg p_i)$ per instance by Lemma 6.4. Summing over $i \in [T]$:

$$\sum_{i=1}^T (O(\lg m \cdot \lambda) + S \lg p_i) = O(m\lambda) + S \sum_{i=1}^T \lg p_i = O(m\lambda + Sm)$$

where the last equality uses $\sum_i \lg p_i = \lg \prod_i p_i = m$.

Combining all steps, the total join-width is $O(m\lambda + Sm)$ as required.

Theorem 5.1 now follows immediately.

Proof of Theorem 5.1. Apply Theorem 6.1 to $S_{\text{aff-}\mathbb{Z}_M}$. The join-width bound from Theorem 6.1 gives garbled program size $O(\lg M(\lambda + S))$ bits. Projectivity holds trivially since the input to the switch system is a control-friendly wire $x \in \{0, 1\}^n$. $\square$

7 Applications

We now explain how to use Theorem 5.3 to obtain projective partial garbling schemes for computing: (1) Argo MAC encodings [EL26] and (2) private scalar multiplication over BN curves [GKS⁺26]. We also formalize the claim in Section 1.2 that our technique immediately implies a non-interactive Minicrypt reduction from large field VOLE to OT with amortized cost $O(\lg p)$ bits.

7.1 Argo MAC Encoding

Argo MAC [EL26] is an elliptic curve homomorphic MAC scheme where the MAC on a point $P \in \mathbb{G}$ is a vector $\sigma(P) + K \in \mathbb{G}^\kappa$. Here, $\sigma : \mathbb{G} \rightarrow \mathbb{G}^\kappa$ is a group homomorphism which can be decomposed into κ many linear endomorphisms $\sigma_i : \mathbb{G} \rightarrow \mathbb{G}$, and $K \in \mathbb{G}^\kappa$. Both σ, K are uniformly sampled by the garbler. To ensure unforgeability of Argo MACs, one must set $\kappa = O(\lambda)$. The group $\mathbb{G} = E(\mathbb{F}_p)$ is the BN elliptic curve group² over $\mathbb{F}_p$, as defined in Appendix B.

Let $\Phi := \{\sigma_i\}_{i \in [\kappa]}$ be the set of κ -tuples of linear endomorphisms on $\mathbb{G}$. Let V denote the subset of affine coordinates corresponding to valid group elements. That is:

$$V = \{(x, y) | P := (x, y) \in E(\mathbb{F}_p)\} \subset \mathbb{F}_p \times \mathbb{F}_p$$

We define the *Argo MAC encoding* function f_{argo} :

$$\begin{aligned} f_{\text{argo}} : V \times (\Phi \times \mathbb{G}^\kappa) &\rightarrow \mathbb{G}^\kappa \\ f_{\text{argo}}((x, y), \sigma, K) &= \sigma(P) + K \quad \text{where } P := (x, y) \in E(\mathbb{F}_p) \end{aligned}$$

Note that this function is defined only on a subset $V \subset \mathbb{F}_p \times \mathbb{F}_p$ (which Definition 1 and Theorem 5.3 allow). In this case, we only guarantee correctness and privacy of our *projectivized* garbling scheme when the encoded input used in evaluation corresponds to an element of V .

Fortunately, the fact that a garbling of f_{argo} does not provide privacy/correctness for all syntactically-valid inputs is not a concern in [EL26]’s proposed application. In their setting, it is G who provides the curve point as input to the garbled computation, and we need not enforce G ’s privacy in the event G is malicious. However, note that if G supplies an invalid input, the resulting MAC could be malformed, preventing E from using it for future computation. To handle this, Argo MAC uses a garbling scheme to *leak an auxiliary secret* to E whenever the input is invalid. Parties separately arrange for a mechanism for the evaluator to punish a cheating G using this auxiliary secret. We refer the reader to [EL26] for more details, noting that these additional checks have negligible cost in the overall scheme.

Implicit in [EL26]’s scheme is an *affine* information-theoretic partial garbling scheme for the function f_{argo} which maps affine coordinates of a valid curve point $(x, y) \in \mathbb{F}_p \times \mathbb{F}_p$ to the κ components of $\sigma(P) + K \in \mathbb{G}^\kappa$. Technically, the scheme outputs (randomized) Jacobian coordinates of the MAC components, which are then reinterpreted as group elements. The key observation is that each coordinate can be computed by a low-degree polynomial over $\mathbb{F}_p$, whose (private) coefficients depend on σ, K . Each polynomial can be privately computed via PGS_{poly} (Theorem 3.1) which is *affine*.

Theorem 7.1 (Argo MAC encoding). *There exists an information-theoretic affine partial garbling scheme PGS_{argo} for computing f_{argo} with $|\hat{f}| = \emptyset$ and $S = O(\kappa)$ encodings over $\mathbb{F}_p$.*

Proof. Implicit in [EL26], Section 2.4 and Section 3.1. □

Applying Theorem 5.3, we obtain a *projective* partial garbling scheme for the function f'_{argo} , modified to accept inputs as bits.

Corollary 7.1. *Assuming CCRH, there exists a projective partial garbling scheme for f'_{argo} (the bitwise-input variant of f_{argo}) with total communication $O(\lambda^2)$ bits.*

Proof. Apply Theorem 5.3 to PGS_{argo} . The input encoding consists of $S = O(\lambda)$ field elements in $\mathbb{F}_p$. With $n := \lceil \lg p \rceil = O(\lambda)$, this gives us total communication $O(Sn + 2\lambda n) = O(\lambda^2)$ bits. □

²[EL26] in fact generalizes to subgroups of $\mathbb{G}$; we restrict to $\mathbb{G}$ itself for clarity of presentation. Our techniques lead to similar improvements in the more general setting.

This is a $O(\lambda)$ factor improvement over the bit-decomposition approach used by [EL26] which incurs total cost $O(\lambda^3)$ bits. Concretely, their protocol (using bit-decomposition) incurs total communication 6.8 MiB for garbling f'_{argo} with $\mathbb{G}$ the BN254 elliptic curve group described in Appendix B and $\kappa = \lceil \lg p / \lg 7 \rceil = 91$ MAC components. In comparison, our approach costs only 355 KiB, a $20\times$ improvement, without increasing computational cost. See Appendix C for details.

7.2 Embryo: Garbled Scalar Multiplication

As explained in Section 4.5, BABE [GKS⁺26] describes how to use witness encryption for linear pairing relations [GHK⁺25] along with a projective partial garbling scheme for private scalar multiplication over a BN curve to verify Groth16 proofs on Bitcoin. Their work first implicitly constructs an affine IT-GS, expressed over $\mathbb{F}_p$, for scalar multiplication. Then, they use bit-decomposition to projectivize the scheme. We now describe how our result implies a more efficient garbling scheme.

In BABE’s setting, E provides a curve point π as input, while G inputs a private scalar r that should remain hidden beyond what is implied by the output $r \cdot \pi$. If E feeds in a point not on the curve, the base IT-GS does not guarantee that r remains hidden. To address this, we incorporate an efficient input validity check directly into our construction: G uses a PRF key derived from a secret ν embedded in the curve-check garbling to mask all scalar-multiplication encodings; E can recover ν —and hence the key—only if her input satisfies the curve equation. This adds only $O(1)$ field elements to the encoding, with negligible effect on cost. In contrast, BABE performs this check using a Yao-style garbled circuit, which incurs unnecessary overhead.

Let $H: \mathbb{F}_p \rightarrow \{0, 1\}^\lambda$ be a random oracle and $\text{PRF}: \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \mathbb{F}_p$ a pseudorandom function. Let $\mathbb{G} = E(\mathbb{F}_p)$ be the BN elliptic curve group over $\mathbb{F}_p$, as defined in Appendix B, and let $q := |\mathbb{G}|$. We define the *scalar multiplication with input validation* function $f_{\text{SM}}: (\mathbb{F}_p \times \mathbb{F}_p) \times \mathbb{F}_q \rightarrow \mathbb{G} \cup \{\perp\}$ as

$$f_{\text{SM}}((\pi_x, \pi_y), r) = \begin{cases} r \cdot \pi & \text{if } \pi := (\pi_x, \pi_y) \in E(\mathbb{F}_p), \\ \perp & \text{otherwise.} \end{cases}$$

That is, the function first checks whether the affine coordinates (π_x, π_y) define a point on the curve, and if so, returns its scalar multiple by r . Following [GKS⁺26]’s framework, we first present an information-theoretic partial garbling scheme for computing f_{SM} . Applying Theorem 5.3, we obtain a *projective* partial garbling scheme for the function f'_{SM} , modified to accept inputs as bits.

For convenience, we define a program $h(\mathbf{p}, \boldsymbol{\pi})$ which evaluates a bivariate polynomial represented by its coefficient vector $\mathbf{p}$ on input $\boldsymbol{\pi} = (\pi_x, \pi_y) \in \mathbb{F}_p^2$. We denote the result by $\mathbf{p}(\boldsymbol{\pi})$. As a subroutine, we use $\text{PGS}_h = (\text{Garb}_h, \text{Enc}_h, \text{Eval}_h)$, the affine partial garbling scheme for Sum of Monomials from Theorem 3.1 for program h .

We start by describing the garbling procedure. The garbler first samples a secret ν and embeds it into a curve-check polynomial q , which evaluates to ν if and only if $\boldsymbol{\pi} \in E(\mathbb{F}_p)$ (and some random value otherwise). It derives a key for a PRF $\text{key} = H(\nu)$ and constructs (randomized) conditional-addition maps $J_{i,j}$ encoding the bits of r . Both q and the $J_{i,j}$ are garbled using PGS_h ; the encoding data of the scalar-multiplication garbling is then masked with PRF outputs keyed by key , so that only an evaluator supplying a valid curve point can recover ν , derive key , and unmask the encodings correctly.

Procedure $\text{Garb}_{\text{SM}}(1^\lambda, r)$:

1. Let $\mathbf{r} = \text{bin}(r)$ denote the bit-decomposition of $r \in \mathbb{F}_q$ i.e. $r = \sum_{i=0}^{n-1} r_i 2^i$ and interpret each bit $r_i \in \{0, 1\} \subset \mathbb{F}_q$ as a field element.
2. Sample $\Delta \in_{\mathbb{S}} \mathbb{F}_p^*$ and $\nu \in_{\mathbb{S}} \mathbb{F}_p$ uniformly. Define the polynomial

$$q(x, y) := (y^2 - x^3 - 3) \cdot \Delta + \nu \in \mathbb{F}_p.$$

Set $\text{key} := H(\nu) \in \{0, 1\}^\lambda$ and initialize a counter $\text{idx} := 0 \in \{0, 1\}^\lambda$.

3. Sample $\phi_i \in_{\mathbb{S}} \mathbb{G}$ uniformly at random subject to $\sum_{i=0}^{n-1} 2^i \phi_i = 0$, and $\lambda_i \in_{\mathbb{S}} \mathbb{F}_p^*$ uniformly at random

for $i \in [n]$. Define polynomial maps $J_i = J_{r_i, \phi_i, \lambda_i} : \mathbb{F}_p^2 \rightarrow \mathbb{F}_p^3$ for $i \in [n]$ as described in Lemma B.5, letting $J_{i,j} : \mathbb{F}_p^2 \rightarrow \mathbb{F}_p$ for $j \in \{1, 2, 3\}$ denote the j -th coordinate map such that $J_{i,j} \in \mathbb{F}_p^{\leq 2}[X, Y]$.

- Let $\mathbf{p}_{i,j}$ denote the coefficient vector of $J_{i,j}$, and let $\mathbf{q}$ denote the coefficient vector of q . For each (i, j) , compute:

$$\text{Garb}_h(1^\lambda, \mathbf{p}_{i,j}) = (\mathbf{a}_{i,j,k}, \mathbf{b}_{i,j,k})_{k \in \{x,y\}}, \quad \mathbf{a}_{i,j,k}, \mathbf{b}_{i,j,k} \in \mathbb{F}_p^{S_{i,j,k}}.$$

For each $k \in \{x, y\}$ and each $\ell \in [S_{i,j,k}]$, set

$$\tilde{\mathbf{b}}_{i,j,k}[\ell] := \mathbf{b}_{i,j,k}[\ell] + \text{PRF}(\text{key}, \text{idx}),$$

incrementing idx after each call. Compute $\text{Garb}_h(1^\lambda, \mathbf{q}) = (\mathbf{a}_{q,k}, \mathbf{b}_{q,k})_{k \in \{x,y\}}$.

Vertically concatenate: $\mathbf{a}_k = \{\mathbf{a}_{i,j,k}\}_{i,j} \parallel \mathbf{a}_{q,k}$ and $\tilde{\mathbf{b}}_k = \{\tilde{\mathbf{b}}_{i,j,k}\}_{i,j} \parallel \mathbf{b}_{q,k}$ for $k \in \{x, y\}$, so that $\mathbf{a}_k, \tilde{\mathbf{b}}_k \in \mathbb{F}_p^{S_k}$ where $S_k := (\sum_{i,j} S_{i,j,k}) + S_{q,k}$.

- Output $\tilde{f} = \emptyset$ and $e = (\mathbf{a}_x, \tilde{\mathbf{b}}_x) \parallel (\mathbf{a}_y, \tilde{\mathbf{b}}_y)$.

Next, we describe the encoding procedure Enc , which is affine over $\mathbb{F}_p$.

Procedure $\text{Enc}_{\text{SM}}(e, \pi)$:

- Parse $e = (\mathbf{a}_x, \tilde{\mathbf{b}}_x) \parallel (\mathbf{a}_y, \tilde{\mathbf{b}}_y)$ and $\pi = (\pi_x, \pi_y)$.
- For $k \in \{x, y\}$, compute $\tilde{\mathbf{c}}_k = \mathbf{a}_k \pi_k + \tilde{\mathbf{b}}_k \in \mathbb{F}_p^{S_k}$.
- Output $\tilde{\pi} = (\tilde{\mathbf{c}}_x, \tilde{\mathbf{c}}_y)$.

Finally, we describe the evaluation procedure. Using the encoded input, the evaluator first evaluates the curve-check polynomial to recover $\hat{\nu}$ and derive a candidate key for the PRF: $\text{key} = H(\hat{\nu})$. She uses key to unmask the remaining encodings, and evaluates the polynomial-evaluation IT-GS to obtain Jacobian coordinates of points $r_i \pi + \phi_i$. Then, she locally computes a linear combination to recover the output $r\pi$.

For clarity, we use blue to denote the output of each step of Eval_{SM} , assuming that $(\tilde{f}, \tilde{\pi})$ is correctly generated by composing Garb_{SM} and Enc_{SM} .

Procedure $\text{Eval}_{\text{SM}}(\tilde{f}, \tilde{\pi})$:

- Parse $\tilde{\pi} = (\tilde{\mathbf{c}}_x, \tilde{\mathbf{c}}_y)$.
- Parse $\tilde{\mathbf{c}}_k = \{\tilde{\mathbf{c}}_{i,j,k}\}_{i \in [n], j \in [3]} \parallel \tilde{\mathbf{c}}_{q,k}$ for $k \in \{x, y\}$, such that $\tilde{\mathbf{c}}_{i,j,k} \in \mathbb{F}_p^{S_{i,j,k}}$ and $\tilde{\mathbf{c}}_{q,k} \in \mathbb{F}_p^{S_{q,k}}$.
- Compute $\hat{\nu} = \text{Eval}_{\text{poly}}(h, \cdot, (\tilde{\mathbf{c}}_{q,x}, \tilde{\mathbf{c}}_{q,y})) = \text{blue} \in \mathbb{F}_p$. Set $\text{key} := H(\hat{\nu}) \in \{0, 1\}^\lambda$ and initialize $\text{idx} := 0$.
- For each (i, j) , $k \in \{x, y\}$, and $\ell \in [S_{i,j,k}]$, unmask:
$$\mathbf{c}_{i,j,k}[\ell] := \tilde{\mathbf{c}}_{i,j,k}[\ell] - \text{PRF}(\text{key}, \text{idx}),$$
 incrementing idx after each call.
- For each (i, j) , compute $\text{Eval}_{\text{poly}}(h, \cdot, (\mathbf{c}_{i,j,x}, \mathbf{c}_{i,j,y})) = v_{i,j} = \text{blue} \in \mathbb{F}_p$.
- For each $i \in [n]$, interpret $(v_{i,1}, v_{i,2}, v_{i,3}) \in \mathbb{F}_p^3$ as Jacobian coordinates of a point $\psi_i = r_i \pi + \phi_i \in \mathbb{G}$.
- Compute $y = \sum_{i=0}^{n-1} 2^i \psi_i = (\sum_{i=0}^{n-1} 2^i r_i) \pi + (\sum_{i=0}^{n-1} 2^i \phi_i) = r\pi \in \mathbb{G}$. Output y .

Lemma 7.1. $\text{PGS}_{\text{SM}} = (\text{Garb}_{\text{SM}}, \text{Enc}_{\text{SM}}, \text{Eval}_{\text{SM}})$ is correct.

Proof. Assume $\pi = (\pi_x, \pi_y)$ corresponds to a valid point $\pi \in \mathbb{G}$. Since $\pi \in E(\mathbb{F}_p)$, we have $q(\pi) = 0 \cdot \Delta + \nu = \nu$, so $\text{key} = H(\nu)$ and the PRF masks cancel: $\mathbf{c}_{i,j,k} = \mathbf{a}_{i,j,k}\pi_k + \mathbf{b}_{i,j,k}$. By correctness of PGS_{poly} (Theorem 3.1), each $v_{i,j} = J_{i,j}(\pi) \in \mathbb{F}_p$. By correctness of $J_{i,j}$ (Lemma B.5), each $\psi_i = r_i\pi + \phi_i \in \mathbb{G}$. Therefore,

$$y = \sum_{i=0}^{n-1} 2^i(r_i\pi + \phi_i) = r \cdot \pi + \sum_{i=0}^{n-1} 2^i\phi_i = r \cdot \pi,$$

where the last equality uses the constraint $\sum_{i=0}^{n-1} 2^i\phi_i = 0$. Note that a semi-honest E holds (π_x, π_y) in the clear so if $\pi \notin \mathbb{G}$ it can output $\perp$ itself, without using the garbled program. $\square$

Lemma 7.2. $\text{PGS}_{\text{SM}} = (\text{Garb}_{\text{SM}}, \text{Enc}_{\text{SM}}, \text{Eval}_{\text{SM}})$ is private.

Proof. We distinguish two cases based on the validity of π .

Case 1: $\pi \in E(\mathbb{F}_p)$.

Procedure $\mathcal{S}_{\text{SM}}(1^\lambda, \pi, r \cdot \pi)$:

1. Sample $\nu \in_{\mathbb{S}} \mathbb{F}_p$ uniformly. Set $\text{key} := H(\nu)$ and $\text{idx} := 0$.
2. For each $i \in [n]$, sample $\psi_i \in_{\mathbb{S}} \mathbb{G}$ uniformly subject to $\sum_{i=0}^{n-1} 2^i\psi_i = r \cdot \pi$. Sample $\lambda_i \in_{\mathbb{S}} \mathbb{F}_p^*$ and compute randomized Jacobian coordinates $\psi_i = (v_{i,1}, v_{i,2}, v_{i,3}) \in \mathbb{F}_p^3$.
3. For each (i, j) , compute $\mathcal{S}_{\text{poly}}(1^\lambda, h, \pi, v_{i,j}) = \{\mathbf{c}_{i,j,k}\}_{k \in \{x,y\}}$. For each $k \in \{x, y\}$ and $\ell \in [S_{i,j,k}]$, set $\tilde{\mathbf{c}}_{i,j,k}[\ell] := \mathbf{c}_{i,j,k}[\ell] + \text{PRF}(\text{key}, \text{idx})$ and increment idx .
4. Compute $\mathcal{S}_{\text{poly}}(1^\lambda, h, \pi, \nu) = \{\tilde{\mathbf{c}}_{q,k}\}_{k \in \{x,y\}}$.
5. Vertically concatenate $\tilde{\mathbf{c}}_k = \{\tilde{\mathbf{c}}_{i,j,k}\}_{i,j} \parallel \tilde{\mathbf{c}}_{q,k}$ for $k \in \{x, y\}$.
6. Output $\tilde{f} = \emptyset$ and $\tilde{\pi} = (\tilde{\mathbf{c}}_x, \tilde{\mathbf{c}}_y)$.

Hyb_1 : The real world: $(\tilde{f}, e) \leftarrow \text{Garb}_{\text{SM}}(1^\lambda, r)$, $\tilde{\pi} \leftarrow \text{Enc}_{\text{SM}}(e, \pi)$.

Hyb_2 : Replace each $\text{Garb}_{\text{poly}}$ with its simulation: for each (i, j) , replace generation of $(a_{i,j,k}, b_{i,j,k})$ and input encoding with $\mathcal{S}_{\text{poly}}(1^\lambda, h, \pi, v_{i,j})$ where $v_{i,j} = J_{i,j}(\pi)$, applying the same PRF masks. Also replace the curve-check garbling with $\mathcal{S}_{\text{poly}}(1^\lambda, h, \pi, \nu)$. This is indistinguishable from Hyb_1 by privacy of PGS_{poly} .

Hyb_3 : Replace each $\psi_i = r_i\pi + \phi_i$ with $\psi_i \in_{\mathbb{S}} \mathbb{G}$ subject to $\sum_{i=0}^{n-1} 2^i\psi_i = r\pi$, and update $v_{i,j}$ used in the above simulations accordingly. This is identically distributed to Hyb_2 since in both hybrids ψ_i are uniform subject to $\sum 2^i\psi_i = r\pi$.

This concludes this case as Hyb_3 is the output of $\mathcal{S}_{\text{SM}}$.

Case 2: $\pi \notin E(\mathbb{F}_p)$.

Procedure $\mathcal{S}_{\text{SM}}(1^\lambda, \pi, \perp)$:

1. Compute $\mathcal{S}_{\text{poly}}(1^\lambda, h, \pi, q(\pi)) = \{\tilde{\mathbf{c}}_{q,k}\}_{k \in \{x,y\}}$.
2. Sample $\tilde{\mathbf{c}}_{i,j,k}[\ell] \in_{\mathbb{S}} \mathbb{F}_p$ independently and uniformly for all (i, j, k, ℓ) .
3. Vertically concatenate $\tilde{\mathbf{c}}_k = \{\tilde{\mathbf{c}}_{i,j,k}\}_{i,j} \parallel \tilde{\mathbf{c}}_{q,k}$ for $k \in \{x, y\}$.
4. Output $\tilde{f} = \emptyset$ and $\tilde{\pi} = (\tilde{\mathbf{c}}_x, \tilde{\mathbf{c}}_y)$.

Hyb_1 : The real world with $\pi \notin E(\mathbb{F}_p)$.

Hyb₂: Replace the curve-check garbling $\text{Garb}_{\text{poly}}$ and input encoding with $\mathcal{S}_{\text{poly}}(1^\lambda, h, \pi, q(\pi))$. This is indistinguishable from Hyb₁ by privacy of PGS_{poly} .

Hyb₃: Replace $H(\nu)$ (used as the PRF key) with a uniformly sampled $\text{key} \in_{\$} \{0, 1\}^\lambda$. After Hyb₂, the value ν appears in the view only as the input to H . Since $\pi \notin E(\mathbb{F}_p)$, we have $q(\pi) = c\Delta + \nu$ where $c := \pi_y^2 - \pi_x^3 - 3 \neq 0$ and $\Delta \in_{\$} \mathbb{F}_p^*$; hence $q(\pi)$ is uniform over $\mathbb{F}_p \setminus \{\nu\}$, statistically independent of ν with error $1/p$. A distinguisher queries H at ν with $\text{negl}(\lambda)$ probability, so this is indistinguishable from Hyb₂.

Hyb₄: Replace each $\text{PRF}(\text{key}, \text{idx})$ with an independent uniform value $\rho_{\text{idx}} \in_{\$} \mathbb{F}_p$. Since key is uniform and independent of the adversary's view, this is indistinguishable from Hyb₃ by PRF security.

Hyb₅: Each scalar-multiplication encoding component $\tilde{\mathbf{c}}_{i,j,k}[\ell] = \mathbf{a}_{i,j,k}[\ell]\pi_k + \mathbf{b}_{i,j,k}[\ell] + \rho_{\text{idx}}$ is uniform in $\mathbb{F}_p$ since ρ_{idx} is uniform and independent. Replace each such component with a fresh $\rho'_{i,j,k,\ell} \in_{\$} \mathbb{F}_p$. This is identically distributed to Hyb₄.

This concludes this case as Hyb₅ is the output of $\mathcal{S}_{\text{SM}}$. $\square$

Theorem 7.2. *In the random oracle model, assuming PRF is a secure pseudorandom function, $\text{PGS}_{\text{SM}} = (\text{Garb}_{\text{SM}}, \text{Enc}_{\text{SM}}, \text{Eval}_{\text{SM}})$ is a statistical affine partial garbling scheme for f_{SM} with $\tilde{f} = \emptyset$ and $S = S_x + S_y = O(n)$ encodings over $\mathbb{F}_p$.*

Proof. Follows from Lemma 7.1 and Lemma 7.2. The encoding function Enc_{SM} is component-wise affine over $\mathbb{F}_p$ by construction. $\square$

Corollary 7.2. *Assuming CCRH and the existence of a random oracle, there exists a projective partial garbling scheme for f'_{SM} (the bitwise-input variant of f_{SM}) with total communication $O(\lambda^2)$ bits.*

Proof. Apply Theorem 5.3 to PGS_{SM} . The input encoding has $S = O(n)$ elements over $\mathbb{F}_p$ with $n = O(\lambda)$, giving communication $O(Sn + 2\lambda n) = O(\lambda^2)$ bits. $\square$

Embryo offers a $O(\lambda)$ factor improvement in communication over the bit-decomposition approach used by BABE which incurs total cost $O(\lambda^3)$ bits. Concretely, BABE requires 22.16 MiB of communication for the same computation, compared to our ≈ 500 KiB (see Appendix C), a $45\times$ improvement.

7.3 $\mathbb{F}_p$ -VOLE Secure Against a Malicious Receiver

We show that our projective garbling scheme for affine evaluation over a large prime field (Theorem 5.2) immediately yields a non-interactive protocol for chosen-input vector oblivious linear evaluation (VOLE) over $\mathbb{F}_p$, secure against a malicious receiver. We work in the $\mathcal{F}_{\text{OT}}$ -hybrid model; the relevant ideal functionalities are given below.

Functionality $\mathcal{F}_{\text{OT}}$

Parties. A sender S and a receiver R .

Inputs. S holds $(m_0, m_1) \in \{0, 1\}^\lambda \times \{0, 1\}^\lambda$; R holds a choice bit $b \in \{0, 1\}$.

Output. R receives m_b . S receives no output.

Functionality $\mathcal{F}_{\text{VOLE}}^S$

Parties. A sender S and a receiver R .

Inputs. S holds $(\mathbf{a}, \mathbf{b}) \in \mathbb{F}_p^S \times \mathbb{F}_p^S$; R chooses a scalar $x \in \mathbb{F}_p$.

Output. R receives $\mathbf{a} \cdot x + \mathbf{b} \in \mathbb{F}_p^S$. S receives no output.

We construct the following protocol Π_{VOLE} in the $\mathcal{F}_{\text{OT}}$ -hybrid model, using the projective garbling scheme $\text{PGS}_{\text{aff-}\mathbb{F}_p} = (\text{Garb}, \text{Enc}, \text{Eval})$ from Theorem 5.2. Here S plays the role of the garbler G and R plays the role of the evaluator E .

Protocol Π_{VOLE} (in the $\mathcal{F}_{\text{OT}}$ -hybrid model)

Sender garbles. S runs $(\tilde{f}, e) \leftarrow \text{Garb}(1^\lambda, (\mathbf{a}, \mathbf{b}))$, where the encoding string takes the form $e = (\Delta, X_0, \dots, X_{\lg p - 1})$ with $\Delta \in \{0, 1\}^\lambda$ and each $X_j \in \{0, 1\}^\lambda$.

OT for encoding. Write $x = \sum_{j=0}^{\lg p - 1} 2^j x_j \in \mathbb{F}_p$ in binary. For each $j \in [\lg p]$, S and R invoke $\mathcal{F}_{\text{OT}}$: S inputs $(X_j, X_j + \Delta)$ and R inputs choice bit x_j . R receives $X_j + x_j \cdot \Delta$. S sends $\tilde{f}$ to R.

Receiver evaluates. R sets $\tilde{x} := (X_j + x_j \Delta)_{j \in [\lg p]} = \text{Enc}(e, x)$ and computes $\text{Eval}(\tilde{f}, \tilde{x})$.

Observe that the OT step is used precisely to secure implement the Enc procedure of $\text{PGS}_{\text{aff-}\mathbb{F}_p}$ —each invocation of $\mathcal{F}_{\text{OT}}$ delivers one projective input label to R. After OT, R sends no further messages and evaluates locally; the protocol is therefore non-interactive beyond the OT calls.

Theorem 7.3 ($\mathbb{F}_p$ -VOLE from OT). *In the $\mathcal{F}_{\text{OT}}$ -hybrid model, assuming the existence of a CCRH, the protocol Π_{VOLE} securely realizes $\mathcal{F}_{\text{VOLE}}^S$ over $\mathbb{F}_p$ against a semi-honest sender and a malicious receiver. The protocol makes $\lg p$ invocations to $\mathcal{F}_{\text{OT}}$ and incurs $O((\lambda + S) \lg p)$ bits of additional communication.*

Proof. The protocol is correct by projectivity and correctness of $\text{PGS}_{\text{aff-}\mathbb{F}_p}$ (Theorem 5.2) because

$$\text{Eval}(\tilde{f}, \text{Enc}(e, x)) = \mathbf{a} \cdot x + \mathbf{b}.$$

Observe that S's view consists solely of his own inputs $(\mathbf{a}, \mathbf{b})$. Formally, semi-honest simulation of S is trivial.

Simulating malicious R is also immediate, since R's only actions are to (1) interact with the OT ideal functionality and (2) receive a garbled encoding, which hides $\mathbf{a}, \mathbf{b}$ by garbling scheme privacy. Formally, the simulator learns R's input x when R tries to invoke the OT functionality. The simulator then sends x to the VOLE functionality and obtains $\mathbf{a}x + \mathbf{b}$. Now, the simulator can use the garbling scheme's simulator to forge a garbling $(\tilde{f}, \tilde{x})$ that outputs $\mathbf{a}x + \mathbf{b}$; it uses $\tilde{x}$ to respond to R's OT queries, and then it sends $\tilde{f}$ to R. The simulator then outputs whatever R outputs. This simulation is indistinguishable from real by privacy of the garbling scheme. $\square$

Communication. The total communication of Π_{VOLE} is $|\tilde{f}| = O(\lg p(\lambda + S))$ bits by Theorem 5.2 plus $\lg p$ invocations of $\mathcal{F}_{\text{OT}}$. Therefore, the marginal cost of generating an OLE correlation is exactly $\lg p$ bits.

Acknowledgements

- We would like to thank Robin Linus for pointing out two small bugs, both of which we have remedied.
- This research was supported by NSF grant CNS-2246353.

References

- [AAK25] Anasuya Acharya, Karen Azari, and Chethan Kamath. On the adaptive security of free-XOR-based garbling schemes in the plain model. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VI*, volume 15606 of *LNCS*, pages 214–244. Springer, Cham, May 2025.
- [ADI⁺17] Benny Applebaum, Ivan Damgård, Yuval Ishai, Michael Nielsen, and Lior Zichron. Secure arithmetic computation with constant computational overhead. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 223–254. Springer, Cham, August 2017.
- [AIK11] Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. How to garble arithmetic circuits. In Rafail Ostrovsky, editor, *52nd FOCS*, pages 120–129. IEEE Computer Society Press, October 2011.
- [App17] Benny Applebaum. Garbled circuits as randomized encodings of functions: a primer. Cryptology ePrint Archive, Report 2017/385, 2017.
- [BCG⁺19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.
- [BCGI18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 896–912. ACM Press, October 2018.
- [BDSW23] Carsten Baum, Samuel Dittmer, Peter Scholl, and Xiao Wang. Sok: vector OLE-based zero-knowledge protocols. *DCC*, 91(11):3527–3561, 2023.
- [BHKO23] Cruz Barnum, David Heath, Vladimir Kolesnikov, and Rafail Ostrovsky. Adaptive garbled circuits and garbled RAM from non-programmable random oracles. Cryptology ePrint Archive, Report 2023/1527, 2023.
- [BHR12a] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Adaptively secure garbling with applications to one-time programs and secure outsourcing. In Xiaoyun Wang and Kazue Sako, editors, *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 134–153. Springer, Berlin, Heidelberg, December 2012.
- [BHR12b] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 784–796. ACM Press, October 2012.
- [BMR90] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In *22nd ACM STOC*, pages 503–513. ACM Press, May 1990.
- [CDK⁺22] Megan Chen, Jack Doerner, Yashvanth Kondi, Eysa Lee, Schuyler Rosefield, abhi shelat, and Ran Cohen. Multiparty generation of an RSA modulus. *Journal of Cryptology*, 35(2):12, April 2022.
- [CFIK03] Ronald Cramer, Serge Fehr, Yuval Ishai, and Eyal Kushilevitz. Efficient multi-party computation over rings. In Eli Biham, editor, *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 596–613. Springer, Berlin, Heidelberg, May 2003.
- [Che25] Weikeng Chen. SoK: BitVM with succinct on-chain cost. Cryptology ePrint Archive, Paper 2025/1253, 2025.
- [CK26] Geoffroy Couteau and Naman Kumar. From ot to ole with almost-linear communication. Cryptology ePrint Archive, Report 2026/295, 2026.

- [CKKZ12] Seung Geol Choi, Jonathan Katz, Ranjit Kumaresan, and Hong-Sheng Zhou. On the security of the “free-XOR” technique. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 39–53. Springer, Berlin, Heidelberg, March 2012.
- [DHIM25] Jack Doerner, Iftach Haitner, Yuval Ishai, and Nikolaos Makriyannis. From OT to OLE with subquadratic communication. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 2249–2263. ACM Press, October 2025.
- [Eag25] Liam Eagen. Glock: Garbled locks for bitcoin. Cryptology ePrint Archive, Paper 2025/1485, 2025.
- [EL26] Liam Eagen and Ying Tong Lai. Argo MAC: Garbling with elliptic curve MACs. Cryptology ePrint Archive, Paper 2026/049, 2026.
- [GHK⁺25] Sanjam Garg, Mohammad Hajiabadi, Dimitris Kolonelos, Abhiram Kothapalli, and Guru-Vamsi Policharla. A framework for witness encryption from linearly verifiable SNARKs and applications. Cryptology ePrint Archive, Report 2025/1364, 2025.
- [Gil99] Niv Gilboa. Two party RSA key generation. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 116–129. Springer, Berlin, Heidelberg, August 1999.
- [GKS⁺26] Sanjam Garg, Dimitris Kolonelos, Mikhail Sergeevitch, Srivatsan Sridhar, and David Tse. BABE: Verifying proofs on Bitcoin made 1000x cheaper. Cryptology ePrint Archive, Report 2026/065, 2026.
- [GLNP18] Shay Gueron, Yehuda Lindell, Ariel Nof, and Benny Pinkas. Fast garbling of circuits under standard assumptions. *Journal of Cryptology*, 31(3):798–844, July 2018.
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016.
- [GS18] Sanjam Garg and Akshayaram Srinivasan. Adaptively secure garbling with near optimal online complexity. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 535–565. Springer, Cham, April / May 2018.
- [GYW⁺23] Xiaojie Guo, Kang Yang, Xiao Wang, Yu Yu, and Zheli Liu. Unmodified half-gates is adaptively secure - so is unmodified three-halves. Cryptology ePrint Archive, Report 2023/1528, 2023.
- [Hea24] David Heath. Efficient arithmetic in garbled circuits. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part V*, volume 14655 of *LNCS*, pages 3–31. Springer, Cham, May 2024.
- [HJO⁺16] Brett Hemenway, Zahra Jafargholi, Rafail Ostrovsky, Alessandra Scafuro, and Daniel Wichs. Adaptively secure garbled circuits from one-way functions. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 149–178. Springer, Berlin, Heidelberg, August 2016.
- [HK21] David Heath and Vladimir Kolesnikov. One hot garbling. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 574–593. ACM Press, November 2021.
- [HvdH21] David Harvey and Joris van der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 193(2):563–617, 2021.
- [IK00] Yuval Ishai and Eyal Kushilevitz. Randomizing polynomials: A new representation with applications to round-efficient secure computation. In *41st FOCS*, pages 294–304. IEEE Computer Society Press, November 2000.

- [IK02] Yuval Ishai and Eyal Kushilevitz. Perfect constant-round secure computation via perfect randomizing polynomials. In Peter Widmayer, Francisco Triguero Ruiz, Rafael Morales Bueno, Matthew Hennessy, Stephan Eidenbenz, and Ricardo Conejo, editors, *ICALP 2002*, volume 2380 of *LNCS*, pages 244–256. Springer, Berlin, Heidelberg, July 2002.
- [IKNP03] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 145–161. Springer, Berlin, Heidelberg, August 2003.
- [IW14] Yuval Ishai and Hoeteck Wee. Partial garbling schemes and their applications. In Javier Esparza, Pierre Fraigniaud, Thore Husfeldt, and Elias Koutsoupias, editors, *ICALP 2014, Part I*, volume 8572 of *LNCS*, pages 650–662. Springer, Berlin, Heidelberg, July 2014.
- [JSW17] Zahra Jafargholi, Alessandra Scafuro, and Daniel Wichs. Adaptively indistinguishable garbled circuits. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part II*, volume 10678 of *LNCS*, pages 40–71. Springer, Cham, November 2017.
- [JW16] Zahra Jafargholi and Daniel Wichs. Adaptive security of Yao’s garbled circuits. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B, Part I*, volume 9985 of *LNCS*, pages 433–458. Springer, Berlin, Heidelberg, October / November 2016.
- [KKPW21] Chethan Kamath, Karen Klein, Krzysztof Pietrzak, and Daniel Wichs. Limits on the adaptive security of Yao’s garbling. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 486–515, Virtual Event, August 2021. Springer, Cham.
- [KMR14] Vladimir Kolesnikov, Payman Mohassel, and Mike Rosulek. FleXOR: Flexible garbling for XOR gates that beats free-XOR. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part II*, volume 8617 of *LNCS*, pages 440–457. Springer, Berlin, Heidelberg, August 2014.
- [Kon25] Yashvanth Kondi. Two-party ECDSA signing at constant communication overhead. Cryptology ePrint Archive, Report 2025/1813, 2025.
- [KS08] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 486–498. Springer, Berlin, Heidelberg, July 2008.
- [LAA⁺25] Robin Linus, Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, Andrea Pelosi, Orfeas Thyfronitis Litos, Christos Stefo, David Tse, and Alexei Zamyatin. Bridging bitcoin to second layers via BitVM2. Cryptology ePrint Archive, Paper 2025/1158, 2025.
- [NPS99] Moni Naor, Benny Pinkas, and Reuban Sumner. Privacy preserving auctions and mechanism design. In *Proceedings of the 1st ACM conference on Electronic commerce*, pages 129–139. ACM, 1999.
- [PSSW09] Benny Pinkas, Thomas Schneider, Nigel P. Smart, and Stephen C. Williams. Secure two-party computation is practical. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 250–267. Springer, Berlin, Heidelberg, December 2009.
- [Roy22] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 657–687. Springer, Cham, August 2022.
- [RR21] Mike Rosulek and Lawrence Roy. Three halves make a whole? Beating the half-gates lower bound for garbled circuits. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 94–124, Virtual Event, August 2021. Springer, Cham.
- [RS21] Peter Rindal and Phillipp Schoppmann. VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 901–930. Springer, Cham, October 2021.

- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.
- [ZRE15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 220–250. Springer, Berlin, Heidelberg, April 2015.

A Hash-Optimized $S_{\text{aff-}\mathbb{Z}_M}$

Recall from Section 6 that each invocation of **word-to-ring** with working modulus $\ell \approx 3 \lg n$ incurs $2^\ell(\ell + 1) = O(n^3 \lg n)$ hash evaluations, giving $\Theta(n^4)$ total across $T = \Theta(n/\lg n)$ primes. This cost is excessive in practice. The culprit is **word-to-hot**, which must enumerate all 2^ℓ possible values of its input. However, in our application we do not need **word-to-ring** for an arbitrary function $g : \mathbb{Z}_{2^\ell} \rightarrow R$ — it suffices to handle functions g that first reduce their input modulo a small integer $p = O(n) \ll 2^\ell$. We exploit this structure to replace the single 2^ℓ -way enumeration with a two-phase process that works over much smaller domains, reducing the total hash count to $O(n^2)$ while preserving the same asymptotic communication. We use this optimization while estimating concrete costs in Appendix C.

Below, we describe a switch system for converting a word label $r \in \mathbb{Z}_{2^\ell}$ where $\ell \leq 3 \lg n$ to a binary OHE h of $r \bmod p$ of length $p \leq n$ with $O(n^2)$ hashes. The construction proceeds in two phases.

Phase 1: Sub-chunk extraction. We decompose the ℓ -bit word r into c sub-chunks by iteratively “peeling off” low-order bits. Let $\ell_0, \ell_1, \dots, \ell_{c-1}$ be bit-widths summing to ℓ , each at most $\lg n$. In our application, $c = 3$. We process sub-chunks from lowest to highest, maintaining a running remainder r that shrinks at each step.

For $q = 0, 1, \dots, c - 1$:

1. **Extract:** Apply the modular reduction homomorphism to obtain a word label for $r^{(q)} = r \bmod 2^{\ell_q} \in \mathbb{Z}_{2^{\ell_q}}$.
2. **Convert:** Apply a modified **word-to-hot** gadget to $r^{(q)}$ that simultaneously produces: (a) binary labels for each bit of $r^{(q)}$, (b) an arithmetic OHE of $r^{(q)}$ with entries in $\mathbb{Z}_{2^\ell}^\lambda$, and (c) a $\mathbb{Z}_{2^\ell}$ word label for $r^{(q)}$. Join-width: $(\ell_q - 1 + \ell)\lambda$ bits. See Remark A.1.
3. **Peel:** If $q < c - 1$, subtract $r := r - r^{(q)}$ in $\mathbb{Z}_{2^\ell}$ (free); the low ℓ_q bits of r are now zero. Apply free division $r := r/2^{\ell_q}$ (a subgroup gate). The remainder r now encodes the upper $\ell - \ell_0 - \dots - \ell_q$ bits.

Output (c) is needed for the subtraction in step 3; output (a) is used in Phase 2; output (b) is used to initialize Phase 2 (for $q = 0$).

After Phase 1, we hold: an arithmetic OHE of $r^{(0)}$ (of length 2^{ℓ_0}), and binary labels for all bits of $r^{(0)}, \dots, r^{(c-1)}$. The total join-width is $\sum_{q=0}^{c-1} (\ell_q - 1 + \ell)\lambda$ bits per prime, and the hash count is $\sum_{q=0}^{c-1} O(2^{\ell_q} \cdot \ell)$ per prime. Since each $\ell_q \leq \lg n$, the per-prime hash count is $O(c \cdot n \cdot \lg n)$, giving $O(n^2)$ total across $T = O(n/\lg n)$ primes.

Remark A.1 (Optimized word-to-hot). *Standard word-to-hot [Hea24] internally composes: (1) bin-to-hot on ℓ_q bits, (2) scale by $1 \in \mathbb{Z}_{2^{\ell_q}}$, and (3) solve-bin (free). This yields outputs (a) and (b) above, but with OHE entries in $\mathbb{Z}_{2^{\ell_q}}^\lambda$ —insufficient for output (c), which requires a $\mathbb{Z}_{2^\ell}$ word label for the subtraction in step 3. Naively obtaining (c) would cost an additional $(\ell_q - 1 + \ell)\lambda$ bits.*

Our modification replaces $1 \in \mathbb{Z}_{2^{\ell_q}}$ with $1 \in \mathbb{Z}_{2^\ell}$ in step (2). The resulting OHE has entries in $\mathbb{Z}_{2^\ell}^\lambda$, so hot-to-word immediately yields a $\mathbb{Z}_{2^\ell}$ word label—all three outputs from a single call at cost $(\ell_q - 1 + \ell)\lambda$.

Phase 2: Folding bits into length p OHE. We combine the sub-chunk bits into a binary OHE h of $r \bmod p$ of length p .

Using Lemma 6.1, we can obtain for free a binary OHE h of $r^{(0)} \bmod p$ of length p . We then fold in the remaining $\ell_1 + \dots + \ell_{c-1}$ bits one at a time. Each fold step transforms an OHE h encoding $v \bmod p$ into an OHE of $(v + b \cdot 2^j) \bmod p$, where b is a binary label at position j :

1. $(s, h') := \text{scale}_1(h)$; $s \boxtimes b$. Define $(1 - b) \cdot h := h - h'$. Cost: p hash evaluations, λ bits.
2. $h_{\text{new}}[r'] := (1 - b) \cdot h[r'] + h'[(r' - 2^j) \bmod p]$.

Since addition modulo p is commutative, the bits may be folded in any order. After all folds, h encodes $r \bmod p$, as required.

The join-width of Phase 2 is $(\ell - \ell_0)\lambda$ bits per prime. The hash count is $(\ell - \ell_0) \cdot p$ per prime, giving $(\ell - \ell_0) \cdot \sum_i p_i = O(n^2)$ total.

Total cost. Combining Phases 1 and 2, the per-prime join-width is

$$\underbrace{\sum_q (\ell_q - 1 + \ell)}_{\text{Phase 1}} + \underbrace{(\ell - \ell_0)}_{\text{Phase 2}} = O(\ell) \text{ bits},$$

yielding $O(\ell\lambda)$ bits of material per prime and $O(n\lambda)$ total across $T = O(n/\lg n)$ primes. The total hash count is $O(n^2)$, with contributions from both Phase 1 and Phase 2.

B Elliptic Curve Preliminaries

In this section, we provide the background required to understand the application of $\text{PGS}_{\text{aff-}\mathbb{F}_p}$ to private scalar multiplication over BN curves, as presented in Section 7.2.

B.1 Notation

We begin by setting up appropriate notation. We follow the notation of [GKS⁺26] for convenience.

- $E/\mathbb{F}_p$ denotes a BN curve in short Weierstrass form

$$E : y^2 = x^3 + 3 \pmod{p},$$

where p is the BN base-field prime. In our applications, $E/\mathbb{F}_p$ is the BN254 curve.

- $\mathbb{G} = E(\mathbb{F}_p)$ denotes the elliptic curve group, consisting of points that satisfy the above equation, equipped with elliptic curve point addition: given two points $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ on E with $P \neq \pm Q$, their sum $P + Q = (x_3, y_3)$ is defined by

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1.$$

For point doubling ($P = Q$), the slope becomes

$$\lambda = \frac{3x_1^2}{2y_1},$$

and x_3, y_3 are computed by the same formulas. The identity element is the point at infinity $\mathcal{O}$.

- $n := \lg p$. In our application, $n = 254$.
- $\mathbb{F}_q$ denotes the finite field of size $q = |\mathbb{G}|$ i.e. the order of the elliptic curve group. For $\mathbb{G}$ equal to BN254, q is a 254-bit prime.
- We use additive group notation, so for $\pi \in \mathbb{G}$ and $r \in \mathbb{F}_q$, the expression $r \cdot \pi$ denotes the group element obtained by adding π with itself r times.
- For a point $\pi \in \mathbb{G} \setminus \{\mathcal{O}\}$, we write $\pi_x, \pi_y \in \mathbb{F}_p$ for its affine coordinates.
- A *Jacobian representation* of π is a triple $(X, Y, Z) \in \mathbb{F}_p^3$ with $Z \neq 0$ satisfying $\pi_x = X/Z^2$ and $\pi_y = Y/Z^3$. This representation is not unique: (X, Y, Z) and $(\lambda^2 X, \lambda^3 Y, \lambda Z)$ represent the same point for any $\lambda \in \mathbb{F}_p^*$. The point at infinity $\mathcal{O}$ is represented by any $(X, Y, 0)$ with $Y \neq 0$. We write $\mathcal{J}(\pi)$ to denote some Jacobian representation of π (not necessarily canonical).

B.2 Elliptic Curve Addition as a Low-Degree Map

We set up lemmas proving that elliptic curve addition can be expressed as a low-degree map over the based field $\mathbb{F}_p$.

Lemma B.1. *Let $\pi = (\pi_x, \pi_y) \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$ be given in affine coordinates, and let $\delta \in \{0, 1\}$. Define the map*

$$J_\delta : \mathbb{F}_p^2 \rightarrow \mathbb{F}_p^3, \quad J_\delta(x, y) := (1 - \delta + \delta x, 1 - \delta + \delta y, \delta).$$

Then $J_\delta(\pi_x, \pi_y) = \mathcal{J}(\delta\pi)$.

Proof. If $\delta = 1$, then $J_\delta(\pi_x, \pi_y) = (\pi_x, \pi_y, 1)$, i.e., the affine point π embedded into Jacobian coordinates. If $\delta = 0$, then $J_\delta(\pi_x, \pi_y) = (1, 1, 0)$, which represents $\mathcal{O}$ under the convention that any $(X, Y, 0)$ with $Y \neq 0$ encodes the point at infinity. $\square$

Lemma B.2 (Point addition [GKS⁺26]). *Fix $\phi = (\phi_x, \phi_y) \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$. For any input $\pi = (x, y) \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$ with $x \neq \phi_x$, define*

$$\begin{aligned} X(x, y) &:= 6 + \phi_x^2 x - 2\phi_y y + \phi_x x^2, \\ Y(x, y) &:= 9\phi_y - (\phi_y^2 + 9)y + 3\phi_x \phi_y x^2 + \phi_y y^2 - 3\phi_x^2 xy, \\ Z(x, y) &:= x - \phi_x. \end{aligned}$$

Then $(X(\pi_x, \pi_y), Y(\pi_x, \pi_y), Z(\pi_x, \pi_y)) = \mathcal{J}(\phi + \pi)$. Moreover, viewed as polynomials in the variables (x, y) (with ϕ_x, ϕ_y fixed), X and Y have total degree 2, and Z has total degree 1.

Lemma B.3 (Conditional point addition). *Fix $\phi = (\phi_x, \phi_y) \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$ and let $\delta \in \{0, 1\}$. For any input $\pi = (x, y) \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$ with $x \neq \phi_x$, define the polynomial map $J_{\delta, \phi} : \mathbb{F}_p^2 \rightarrow \mathbb{F}_p^3$ by*

$$J_{\delta, \phi}(x, y) := (X_{\delta, \phi}(x, y), Y_{\delta, \phi}(x, y), Z_{\delta, \phi}(x, y)),$$

where

$$\begin{aligned} X_{\delta, \phi}(x, y) &:= (1 - \delta)\phi_x + \delta(6 + \phi_x^2 x - 2\phi_y y + \phi_x x^2), \\ Y_{\delta, \phi}(x, y) &:= (1 - \delta)\phi_y + \delta(9\phi_y - (\phi_y^2 + 9)y + 3\phi_x \phi_y x^2 + \phi_y y^2 - 3\phi_x^2 xy), \\ Z_{\delta, \phi}(x, y) &:= (1 - \delta) + \delta(x - \phi_x). \end{aligned}$$

Then $J_{\delta, \phi}(\pi_x, \pi_y) = \mathcal{J}(\phi + \delta\pi)$. Moreover, $J_{\delta, \phi}$ has degree 2 as a polynomial map.

Proof. Immediate from Lemmas B.1 and B.2. $\square$

Lemma B.4 (Jacobian randomization [GKS⁺26]). *Given a point $\pi \in E(\mathbb{F}_p)$ in Jacobian coordinates (X, Y, Z) , we can obtain a uniform representation of the same point as $\text{diag}(\lambda^2, \lambda^3, \lambda) \cdot (X, Y, Z)^\top$ for λ sampled uniformly from $\mathbb{F}_p^*$, where*

$$\text{diag}(\lambda) = \begin{pmatrix} \lambda^2 & 0 & 0 \\ 0 & \lambda^3 & 0 \\ 0 & 0 & \lambda \end{pmatrix}.$$

Lemma B.5 (Randomized conditional point addition). *Fix $\phi \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$, $\delta \in \{0, 1\}$, and $\lambda \in \mathbb{F}_p^*$. Define $J_{\delta, \phi, \lambda} : \mathbb{F}_p^2 \rightarrow \mathbb{F}_p^3$ by*

$$J_{\delta, \phi, \lambda}(x, y) := \text{diag}(\lambda^2, \lambda^3, \lambda) \cdot J_{\delta, \phi}(x, y).$$

Then:

1. $J_{\delta, \phi, \lambda}(\pi_x, \pi_y) = \mathcal{J}(\phi + \delta\pi)$ for any $\pi = (x, y) \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$ with $x \neq \phi_x$.
2. If λ is uniform over $\mathbb{F}_p^*$, then $J_{\delta, \phi, \lambda}(\pi_x, \pi_y)$ is uniformly distributed among all Jacobian representations of $\phi + \delta\pi$.
3. $J_{\delta, \phi, \lambda}$ has degree 2 as a polynomial map in (x, y) .

Proof. Immediate from Lemmas B.3 and B.4. $\square$

C Concrete Cost Analysis

In this section, we provide a concrete cost estimate for our scheme **Embryo** which we presented in Section 7.2. We estimate total communication as well as number of hash invocations, which we assume will dominate computational costs. We compare our results to BABE [GKS⁺26] and report on our improvement in communication, observing that we achieve this improvement *without increasing computation*. We also comment on the concrete cost of our scheme used to encode Argo MACs (Section 7.1) and compare it to [EL26].

Parameters. For simplicity, and conservatively, let $n = 256$ so that $\lg n = 8$ and $n/\lg n = 32$. We set the computational security parameter $\lambda = 128$ bits. Let p_i denote the i -th prime, where $p_1 = 2$. Using the first 80 primes is sufficient to support our operation:

$$p_{\max} = p_{80} = 409 \quad \left(\sum_{i=1}^{80} \log_2 p_i \right) > 552 \quad \left(\sum_{i=1}^{80} \lg p_i \right) = \left(\sum_{i=1}^{80} \lceil \log_2 p_i \rceil \right) = 593 \quad \left(\sum_{i=1}^{80} p_i \right) = 14,697$$

Let M denote the product of these primes. We split the n -bit input into $C = n/\lg n = 32$ chunks of $\lg n = 8$ bits each. The working modulus ℓ must be large enough to support C values each at most $n - 1$, scaled by at most $p_{\max} - 1$:

$$C \cdot (n - 1) \cdot (p_{\max} - 1) = 3,329,280 < 2^{22},$$

so we set $\ell = 22$.

Step 1: Chunk Conversion. We apply bin-to-word to each of the $C = 32$ chunks, converting each 8-bit chunk to a control-friendly word in $\mathbb{Z}_{2^{22}}$. Each instance of bin-to-word is invoked with input length $\lg n = 8$ and output ring $\mathbb{Z}_{2^\ell} = \mathbb{Z}_{2^{22}}$, contributing join-width $(\lg n - 1 + \ell)\lambda = 29\lambda$ by Lemma 6.3. Concretely, the $\lg n - 1 = 7$ term comes from bin-to-hot on the 8-bit chunk, and the $\ell = 22$ term comes from scale-hot by $1 \in \mathbb{Z}_{2^{22}}$. Each instance incurs $2^{\lg n} - 2 = 254$ hashes from bin-to-hot and $2^{\lg n} \cdot \ell = 256 \cdot 22 = 5,632$ hashes from scale-hot, totaling 5,886 hashes. Summing over all 32 chunks:

$$32 \cdot 29\lambda = 928\lambda \approx 14.5 \text{ KiB communication}, \quad 32 \cdot 5,886 \approx 188\text{K hashes}.$$

Step 2: Free Accumulation into CRT Residues. Step 2 of $\mathcal{S}_{\text{aff-}\mathbb{Z}_M}$ consists entirely of homomorphism gates (constant multiplication and addition on $\mathbb{Z}_{2^\ell}$ labels) and incurs no communication or hashing cost.

Step 3: Residue Evaluation. We apply the hash-optimized word-to-ring $_{g_i}$ (Appendix A) to each of the $T = 80$ residue wires $r_i \in \mathbb{Z}_{2^{22}}$, with $R = \mathbb{Z}_{p_i}^S$. Each invocation proceeds in two phases.

1. **Sub-chunk extraction.** We decompose each 22-bit residue wire r_i into three sub-chunks: low (8 bits), middle (7 bits), and high (7 bits).

For the low 8 bits, we apply the modified word-to-hot gadget of Appendix A, scaling internally by $1 \in \mathbb{Z}_{2^{22}}$ rather than $1 \in \mathbb{Z}_{2^8}$, at cost:

$$(8 - 1 + \ell)\lambda = 29\lambda \text{ communication}, \quad (2^8 - 2) + 2^8 \cdot 22 = 5,886 \text{ hashes}.$$

We then subtract and free-divide to isolate the middle 7 bits, repeating the gadget at cost $(7 - 1 + \ell)\lambda = 28\lambda$ and 2,942 hashes, and likewise for the high 7 bits. The total cost of this phase per residue is:

$$(29 + 28 + 28)\lambda = 85\lambda \text{ communication}, \quad 5,886 + 2,942 + 2,942 = 11,770 \text{ hashes}.$$

2. **Folding bits into the OHE.** Now we hold a one-hot encoding of the low 8 bits (of length 256) and binary labels for the remaining 14 middle and high bits. We first reduce the low-bit OHE modulo p_i to obtain a length- p_i OHE of $[r_i]_{p_i}$, then fold in the remaining 14 bits one at a time. Each fold step incurs p_i hashes and λ bits of communication. The total cost of this phase per residue is:

$$14\lambda \text{ communication}, \quad 14 \cdot p_i \text{ hashes}.$$

Summing the hash cost over all 80 primes: $14 \cdot \sum_{i=1}^{80} p_i = 14 \cdot 14,697 = 205,758$ hashes.

Combining Phases 1 and 2, the cost per residue is $(85 + 14)\lambda = 99\lambda$ communication and $11,770 + 14 \cdot p_i$ hashes. Summing over all 80 residues:

$$80 \cdot 99\lambda = 7,920\lambda \approx 124 \text{ KiB communication}, \quad 80 \cdot 11,770 + 205,758 \approx 1.15\text{M hashes}.$$

The cost up till this point is a *one time CRT-conversion* cost i.e. it scales only with the number of inputs being encoded but *not* with the complexity of the function being evaluated (and therefore dimension of affine encodings).

IT-GS Label Cost. We start by counting the dimension of the encodings in the IT-GS (which is affine over $\mathbb{F}_p$) used to compute f_{SM} in Section 7.2. Observing the polynomials in Lemma B.5 and using Theorem 3.1, we conclude that the total dimension of encodings (across both inputs x and y) is 12: Four non-zero entries are degree-1 functions of the affine coordinates x, y and four non-zero entries are degree-2 functions of the affine coordinates x, y .

Each CRT ring element has size $\sum_{i=1}^{80} \lg p_i = 593$ bits. Since there are $n = 256$ maps ϕ_i , the total cost of IT-GS labels is:

$$256 \cdot (593 \cdot 12) \text{ b} = 1,821,696 \text{ b} \approx 222.5 \text{ KiB communication}.$$

The IT-GS label output uses **scale-hot** with a non-control-friendly scalar. Each prime p_i contributes p_i switches, each hashing a λ -bit control label to $D \cdot \lg p_i$ bits, where $D = 12 \cdot 256 = 3,072$. Since one hash call outputs $\lambda = 128$ bits, each switch requires $\lceil D \cdot \lg p_i / \lambda \rceil$ calls. For $\lg p_i \leq 9$, this is at most $\lceil 3,072 \cdot 9 / 128 \rceil = 216$ calls per switch. Summing conservatively over all primes:

$$14,697 \times 216 \approx 3.17\text{M hashes}.$$

Curve Check. Given affine coordinates x, y written in binary, we check the elliptic curve equation $y^2 = x^3 + 3$ via an IT-GS of size 5 ring elements:

$$(593 \cdot 5) \text{ b} \approx 0.5 \text{ KiB}.$$

Embryo Total Cost. We must compute the Boolean to CRT conversion twice, once for each coordinate x and y . The total costs are summarized in the table below.

Component	Communication	Hash calls
Step 1 (chunk conversion) $\times 2$	$2 \times 14.5 \text{ KiB}$	$2 \times 188\text{K}$
Step 2 (free accumulation) $\times 2$	—	—
Step 3 (residue evaluation) $\times 2$	$2 \times 124 \text{ KiB}$	$2 \times 1.15\text{M}$
IT-GS labels	222.5 KiB	3.17M
Curve check	0.5 KiB	—
Total	= 500 KiB	= 5.8M

Comparison to BABE. BABE [GKS⁺26] requires 22.16 MiB of communication for this task. On the other hand, we achieve total communication $\approx 500 \text{ KiB}$, a $45\times$ improvement.

On computation, BABE reports a runtime exceeding 100 ms. Our scheme invokes $\approx 5.8 \text{ M}$ hash calls; instantiated with fixed-key AES as the underlying primitive, this amounts to under 25 ms. While our scheme incurs additional costs beyond hashing — including arithmetic operations and CRT reconstruction — we expect these to be dominated by the hash cost in practice. We therefore conclude that our scheme achieves a $45\times$ reduction in communication *without increasing computation*.

Argo MAC Encoding Cost. The above methodology extends naturally to computing the cost of garbling Argo MAC encodings [EL26] for the group of points over the BN254 curve (described in Appendix B). The fixed cost of converting two coordinates to CRT representation is identical to the scalar multiplication setting: Steps 1 and 2 of $S_{\text{aff-ZM}}$ are unchanged, and the shape of the polynomials evaluated in Step 3 also happens to be the same.

The key difference is in the number of polynomials evaluated. In the scalar multiplication setting, $n = \lg p = 254$ maps ϕ_i are required: one per bit of the scalar. Argo MACs are parametrized by the number of MAC components generated for an elliptic curve point. For the intended application of garbling a Groth16 verifier, [EL26] claims that it suffices to use

$$\kappa = \left\lceil \frac{\lg p}{\lg_2 7} \right\rceil = \left\lceil \frac{254}{2.807} \right\rceil = 91$$

components for one point of the Groth16 proof. This requires evaluating $\kappa = 91$ polynomials rather than 254. The IT-GS label cost is modified accordingly: we set $D = 12 \cdot \kappa = 1,092$, which reduces the hash calls per switch to

$$\left\lceil \frac{1,092 \cdot 9}{128} \right\rceil = 77,$$

and the communication to $\kappa \cdot 593 \cdot 12 \text{ b} \approx 77.3 \text{ KiB}$. The costs are summarized below.

Component	Communication	Hash calls
Steps 1–3 (CRT conversion) $\times 2$	$2 \times 138.5 \text{ KiB}$	$2 \times 1.34\text{M}$
IT-GS labels ($\kappa = 91$)	77.3 KiB	1.13M
Total	= 355 KiB	= 3.81M

[EL26] report a garbled circuit size of 6.8 MiB for this computation. On the other hand, we achieve total communication $\approx 355 \text{ KiB}$, a $20\times$ improvement.